Implementing and Empirically Evaluating the Mixture-of-Superpositions-based Protocol for Classically Verifiable Quantum Learning

Alex Do
Student ID: 5505454

Supervisor: Dr. Matthias Caro

April 2026

Department of Computer Science
University of Warwick

Word count: 6210

Abstract

TODO: derive an abstract

Keywords: Quantum Verification, Mixture of Superpositions, Fourier Learning, Distribution-Agnostic learning, Boolean Function Learning, Classical Verification

Acknowledgements

I am deeply grateful to my supervisor, Dr. Matthias Caro, whose patient guidance throughout the academic year has shaped the report at every stage. Working on a protocol with one of its own authors has been a privilege I wish I made more use of. I also thank Dr. Christian Ikenmeyer, my second assessor, for his strong and thorough feedback on my Progress Report submission, of which had a significant impact on my communication style of this report. Finally, I thank my family and friends for their continued support throughout my degree, especially during a busy final year.

Contents

1 Introduction . . . 1

1.1 Motivation . . . 1

1.2 Research Questions . . . 1

1.3 Objectives and Contributions . . . 1

1.4 Report Structure . . . 1

2 Background . . . 2

2.1 Quantum Preliminaries . . . 2

2.1.1 Dirac Notation for Quantum Mechanics . . . 2

2.1.2 Qubits and State Vectors . . . 2

2.1.3 Quantum Gates . . . 3

2.1.4 Measurement . . . 4

2.2 Interactive Proof Systems . . . 4

2.2.1 Beyond Language Recognition . . . 5

2.3 Functional and Distributional-Agnostic Learning . . . 5

2.4 Fourier Analysis for Classical and Quantum Learning . . . 6

2.4.1 Parity Basis and Fourier Coefficients . . . 6

2.4.2 Quantum Fourier Sampling for Parity . . . 7

2.5 Classically Verifying Quantum Learning . . . 8

3 Mixture-of-Superpositions-based Verification Protocol . . . 9

3.1 Mixture-of-Superpositions Construction . . . 9

3.1.1 Approximate Quantum Fourier Sampling from MoS . . . 10

3.2 The Four-Step Protocol . . . 10

3.3 Protocol Parameters (INCOMPLETE, CHECK) . . . 11

4 Implementation of the Mixture-of-Superpositions Verification Protocol . . . 14

4.1 Architecture Overview . . . 14

4.2 Mixture-of-Superpositions Simulator . . . 14

4.2.1 Implementation of Fourier analysis over MoS . . . 18

4.2.2 Sampling from `MoSState` . . . 19

4.3 Quantum Prover . . . 20

4.4 Classical Verifier . . . 21

5 Experiments and Results ... 23
5.1 Completeness Experiments ... 23
5.1.1 Scaling baseline ... 24
5.2 Soundness Experiments ... 25
5.2.1 Single-parity dishonest provers ... 25
5.2.2 Multi-element dishonest provers ... 26
5.3 Robustness Experiments ... 27
5.3.1 Label-flip noise sweep ... 27
5.3.2 Gate-level depolarising noise ... 29
5.4 Sensitivity and Practical Limitations ... 30
5.4.1 Bent functions ... 30
5.4.2 θ-sensitivity ... 31
6 Discussion of Experiments ... 33
6.1 Alignment with known theory and bounds ... 33
6.2 Protocol failure modes ... 33
6.3 Gate noise as an open frontier ... 33
6.4 Limitations ... 33
7 Conclusions and Future Work ... 34
7.1 Summary of Contributions ... 34
7.2 Future Work ... 34
A MoSState Class - An implementation of the MoS concept from Caro et al. [1] ... 35
A.1 State constructor ... 35
A.2 Property accessors ... 36
A.3 $f \sim F_{\mathcal{D}}$ sampler ... 36
A.4 State preparation ... 37
A.5 Density matrix of $\rho_{\mathcal{D}}$ ... 38
A.6 Classical sampling mechanisms ... 38
A.7 Fourier analysis tools ... 39
B The `QuantumFourierSampler` for sampling from MoS ... 40
B.1 Constructor ... 40
B.2 Sampling mechanism ... 41
B.2.1 Inner sampling mechanism ... 42

B.3 QFSResult class 44
C Implementation of the Prover from Caro et al. [1] 45
C.1 Approximation result class 45
C.2 Prover's message to the verifier 45
C.3 Constructor of `MoSProver` 46
C.4 The central running protocol for sampling and aggregation of $\rho_{\mathcal{D}}$ 47
C.5 Building a Fourier spectrum approximation 48
C.6 Estimating and extracting heavy coefficients 48
D Implementation of the Verifier from Caro et al. [1] 50
D.1 Verification outcomes and hypothesis types 50
D.2 Parity hypothesis for 1-agnostic learning 50
D.3 Fourier-k-sparse hypothesis for 2-agnostic learning 51
D.4 Verification result 52
D.5 MoSVerifier Constructor 52
D.6 General verifier logic 53
D.7 Verifier coefficient estimation 54
D.8 Building hypotheses 55
E Circuit diagram of sampling MoS with $n = 3$ 56
Bibliography 57

1 Introduction

1.1 Motivation

1.2 Research Questions

1.3 Objectives and Contributions

1.4 Report Structure

2 Background

2.1 Quantum Preliminaries

2.1.1 Dirac Notation for Quantum Mechanics

The bra-ket notation [2] introduced by Paul Dirac in 1939 provides a concise algebraic framework for the manipulation of vectors and linear functionals within a Hilbert space $\mathcal{H}$. Throughout this report, all Hilbert spaces are finite-dimensional and are isomorphic to $\mathbb{C}^{2^n}$ for some $n \in \mathbb{N}$, a complex vector space equipped with an inner product.

> **Definition 1 (Bra-ket notation)**. A ket vector $|\psi\rangle$ is an element of the Hilbert space $\mathcal{H}$, representing a quantum state. In a finite-dimensional basis, this corresponds to a column vector in $\mathbb{C}^d$. A bra vector $\langle\phi|$ is an element of the dual space $\mathcal{H}^*$, obtained as the Hermitian conjugate transpose of its corresponding ket: $\langle\phi| = (|\phi\rangle)^\dagger$.

The inner product $\langle\varphi|\psi\rangle$ of two states $|\varphi\rangle, |\psi\rangle \in \mathcal{H}$ yields a scalar in $\mathbb{C}$. The outer product $|\varphi\rangle\langle\psi|$ is the rank-1 linear operator on $\mathcal{H}$ defined by its action on an arbitrary state $|\chi\rangle \in \mathcal{H}$:

$$|\varphi\rangle\langle\psi|)|\chi\rangle = \langle\psi|\chi\rangle \; |\varphi\rangle$$

2.1.2 Qubits and State Vectors

A *qubit* is the fundamental unit of quantum information, analogous to a classical bit. Mathematically, the state of a single qubit is a unit vector in the two-dimensional complex Hilbert space $\mathbb{C}^2$.

> **Definition 2 (Computational basis)**. The *computational basis* of $\mathbb{C}^2$ consists of two orthonormal column vectors:
>
> $$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$
>
> An arbitrary single-qubit state can be written as a *superposition*:
>
> $$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad \alpha, \beta \in \mathbb{C}, \qquad |\alpha|^2 + |\beta|^2 = 1.$$
>
> The constraint $|\alpha|^2 + |\beta|^2 = 1$ is the *normalisation condition*, ensuring that $|\psi\rangle$ is a unit vector under the standard inner product $\langle\phi|\psi\rangle = \sum_i \overline{\phi_i}\psi_i$.

The state space of an n-qubit system is constructed via tensor product. Concretely, n qubits inhabit the Hilbert space:

$$(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$$

Which has dimension 2^n. The computational basis of this space is indexed by n-bit strings:

$$\{|x\rangle : x \in \{0,1\}^n\}$$

Where for example $|x_1, ..., x_n\rangle = |x_1\rangle \otimes ... \otimes |x_n\rangle$. A general n-qubit pure state takes the form:

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \qquad \sum_x |\alpha_x|^2 = 1$$

In the protocol of Section 4, the prover prepares $(n+1)$-qubit registers: n qubits encoding the input domain $\{0,1\}^n$ and one ancilla qubit used for postselection (see Section 2.1.4).

2.1.3 Quantum Gates

A quantum gate acting on k qubits is a $2^k \times 2^k$ unitary matrix U (where $U^\dagger U = \mathbb{I}$). Unitarity guarantees that gate application preserves the normalisation condition and is reversible. We introduce only the gates that appear in the verification protocol.

Pauli-X gate. The Pauli-X gate is the quantum analogue of a classical NOT gate, applied to a single qubit and swapping $|0\rangle \leftrightarrow |1\rangle$:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Hadamard gate. The Hadamard gate maps computational basis states to uniform superpositions over a single qubit:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Its action on the computational basis is, for $b \in \{0,1\}$:

$$H|b\rangle = \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} (-1)^{b \cdot y} \; |y\rangle$$

***n*-fold Hadamard.** Applying H independently to each of n qubits yields $H^{\otimes n}$, which acts on computational basis states as:

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} \; |y\rangle$$

Where $x \cdot y = \bigoplus_{i=1}^n x_i y_i$ denotes the bitwise inner product modulo 2. This is the discrete analogue of the Fourier transform over $\mathbb{F}_2^n$ and is central to the Quantum Fourier Sampling (QFS) subroutine in Section 2.4.

Multi-controlled-X (MCX) Gate. The MCX gate flips a designated target qubit if and only if all n control qubits are in state $|1\rangle$. In this protocol, MCX gates are used within an oracle U_f that encodes a boolean function $f : \{0,1\}^n \to \{0,1\}$ via:

$$U_f|x\rangle|b\rangle = |x\rangle|b \oplus f(x)\rangle$$

The MCX gate is central to the implementation we provide in Section 4.

2.1.4 Measurement

Definition 3 (Born rule). The fundamental measurement postulate states that measuring an n-qubit state $|\psi\rangle = \sum_x a_x|x\rangle$ yields outcome $x \in \{0,1\}^n$ with probability:

$$\mathbb{P}[\text{outcome } x] = |a_x|^2$$

And the post-measurement state collapses to $|x\rangle$.

When only a subset of n qubits is measured, the remaining qubits are left in a conditional state determined by the measurement outcome. Consider an $(n+1)$-qubit state written as:

$$|\psi\rangle = \sum_{x\in\{0,1\}^n} \sum_{b\in\{0,1\}} \alpha_{x,b}\ |x\rangle\ |b\rangle$$

Measuring the last qubit and obtaining outcome b_0 occurs with probability:

$$p_{b_0} = \sum_x |a_{x,b_0}|^2$$

And the conditional (post-measurement) state of the first n qubits is:

$$|\psi_{b_0}\rangle = \frac{1}{\sqrt{p_{b_0}}} \sum_x a_{x,b_0}\ |x\rangle \tag{1}$$

Definition 4 (Postselection). *Postselection* is the procedure of discarding all runs of a quantum computation except those in which a designated measurement yields a specific outcome. The resulting (accepted) state is the conditional state in Equation 1 corresponding to that outcome.

In the construction of Caro et al.'s classical verification for quantum learning protocol [1] (which we implement in Section 4.3), the prover prepares an $(n+1)$-qubit state and measures the ancilla qubit. Only runs yielding outcome $b = 1$ are retained, the measurement shapes the amplitude distribution of the remaining n-qubits into one that encodes a desired Fourier spectrum.

2.2 Interactive Proof Systems

An interactive proof system formalises the idea that a computationally weak party (the *verifier*) can become convinced of the truth of a statement by engaging in a dialogue with a powerful party (the *prover*). The notion was introduced by Goldwasser, Micali, and Rackoff [3].

Definition 5 (Interactive proof system). An *interactive proof system* for a language L is a protocol between a computationally unbounded prover P and a probabilistic polynomial-time verifier V. After exchanging a polynomial number of messages on common input x, the verifier outputs *accept* or *reject*. The protocol must satisfy:

1. **Completeness:** for each k and for sufficiently large $x \in L$, the verifier V halts and accepts with probability at least $1 - |x|^{-k}$.
2. **Soundness:** for each k and for sufficiently large $x \notin L$, for any cheating prover P', the verifier V accepts with probability at most $|x|^{-k}$.

The complexity class *NP* can be viewed as a degenerate interactive proof: the prover sends a single certificate and the verifier checks it *deterministically* and in polynomial time with perfect soundness. Allowing the verifier to use randomness and to exchange multiple rounds of messages with the prover yields a more powerful proof model. The resulting complexity class *IP*, the set of all languages admitting an interactive proof system, was shown by Shamir [4] to equal *PSPACE*. Thus:

$$\mathrm{NP} \subseteq \mathrm{IP} = \mathrm{PSPACE}$$

2.2.1 Beyond Language Recognition

Definition 5 is stated for language membership, but the IP paradigm extends naturally to settings where the claim being verified is not "$x \in L$" but rather "a computation was performed correctly".

The protocol of Caro et al. [1] is an instance of this pattern. The claim under verification is that a quantum learner has produced a hypothesis that approximates a target Boolean function f. The prover is a bounded-error quantum polynomial-time (BQP) machine and the verifier is entirely classical (BPP). In their protocol, the verifier issues challenges and checks the prover's responses against classical samples, which we will explore in Section 2.5.

2.3 Functional and Distributional-Agnostic Learning

This section establishes the learning-theoretic foundations underlying the classical verification of quantum learning framework from Caro et al. [1]. We begin by formalising the case of the agnostic learning task [5]. There are two canonical choices documented:

- In *functional-agnostic* learning with respect to uniformy random inputs, we assume the data consists of labelled inputs $(x_i, f(x_i))$ with x_i drawn i.i.d. uniformly at random from $\mathcal{X} = \{0,1\}^n$ and $f : \{0,1\}^n \to \{0,1\}$, an unknown boolean function. We denote the data-generating distribution as $\mathcal{D} = (\mathcal{U}_n, f)$.
- In the *distributional-agnostic* setting, with respect to uniformy random inputs, we no longer assume a perfectly descriptive f. In other words we assume samples labelled (x_i, y_i) are drawn i.i.d. over some distribution $\mathcal{D}$ over $\{0,1\}^n \times \{0,1\}$ with uniform marginal over $\{0,1\}^n$. We denote this as $\mathcal{D} = (\mathcal{U}_n, \varphi)$ where φ:

$$\varphi(x) = \mathbb{P}_{(x,y)\sim\mathcal{D}}[y = 1 \mid x]$$

The goal in both types of learning is to learn an almost-optimal approximating function compared to a benchmark class $\mathcal{B}$, such that given an accuracy parameter ε, confidence parameter δ, and access to training data i.i.d. from $\mathcal{D}$, an α-agnostic learner has to output, with success probability $\geq 1-\delta$ a hypothesis h such that:

$$\mathbb{P}_{(x,y)\sim\mathcal{D}}[h(x) \neq y] \leq \alpha \cdot \inf_{b\in\mathcal{B}} \mathbb{P}_{(x,y)\sim\mathcal{D}}[b(x) \neq y] + \varepsilon$$

In simpler words, the hypothesis is at most as incorrect as the best benchmark (by metric of accuracy) for any input x, multiplied by α with some additive error margin ε.

In quantum learning theory, a learner can have access the distribution $\mathcal{D}$ with training data canonically taken to consist of copies of the *quantum superposition example state* [6]:

$$|\phi_{\mathcal{D}}\rangle = \sum_{(x,y)\in\{0,1\}^n\times\{0,1\}} \sqrt{\mathcal{D}(x,y)}|x,y\rangle \tag{2}$$

Whilst the above data is at least as powerful as its classical counterpart (since we can simulate classical data via computational basis measurements), it is unknown how to use copies of $|\phi_{\mathcal{D}}\rangle$ to improve upon classical distributional-agnostic learning. Caro et al. [1] present a new idea of *mixture-of-superpositions* (MoS) to attempty to overcome this.

2.4 Fourier Analysis for Classical and Quantum Learning

Where Fourier analysis is typically performed over periodic functions, here the domain is the finite set $\{0,1\}^n$, the Boolean hypercube. Let $f : \{0,1\}^n \to \{0,1\}$, a Boolean function. It is often convenient to work with the equivalent $\{-1,1\}$-valued representation $g : \{0,1\}^n \to \{-1,1\}$ obtained via the bijection $g(x) = (-1)^{f(x)} = 1 - 2f(x)$. This signed representation interacts cleanly with algebraic structure and will be used throughout.

The space of all real-valued functions on $\{0,1\}^n$ is equipped with the inner product:

$$\langle\phi,\psi\rangle = \mathbb{E}_{x\sim\mathcal{U}_n}[\phi(x)\psi(x)] = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} \phi(x)\psi(x)$$

Where $\mathcal{U}_n$ denotes the uniform distribution over $\{0,1\}^n$ and makes the function space a 2^n-dimensional Hilbert space over $\mathbb{R}$.

2.4.1 Parity Basis and Fourier Coefficients

For each $s \in \{0,1\}^n$, define the *parity function*:

$$\chi_s : \{0,1\}^n \to \{-1,1\}, \qquad \chi_{s(x)} = (-1)^{s\cdot x}$$

Definition 6 (Fourier coefficients and expansion). Let $\phi : \{0,1\}^n \to \mathbb{R}$. The Fourier coefficient of ϕ at frequency $s \in \{0,1\}^n$ is:

$$\hat{\phi}(s) = \langle \phi, \chi_s \rangle = \mathbb{E}_{x \sim \mathcal{U}_n}[\phi(x)\chi_s(x))]$$

Since $\{\chi_s\}_{s \in \{0,1\}^n}$ forms an orthonormal basis for $\mathbb{R}^{\{0,1\}^n}$, ϕ decomposes uniquely as $\phi = \sum_{s \in \{0,1\}^n} \hat{\phi}(s)\chi_s$. In other words, every function on the Boolean hypercube can be expressed as a unique linear combination of parity functions.

Following this, we can state a fundamental identity relating the ℓ^2-norm of a function to its Fourier coefficients:

Theorem 1 (Parseval's Identity). *For any $\phi : \{0,1\}^n \to \mathbb{R}$:*

$$\mathbb{E}_{x \sim \mathcal{U}_n}[(\phi(x))^2] = \sum_{s \in \{0,1\}^n} \left(\hat{\phi}(s)\right)^2$$

Note that if ϕ is $\{-1,1\}$-valued, then $\sum_{s \in \{0,1\}^n} \left(\hat{\phi}(s)\right)^2 = 1$, so the squared Fourier coefficients form a probability distribution over $\{0,1\}^n$.

Proof. See [7]. □

The collection $\left\{\hat{\phi}(s)\right\}_{s \in \{0,1\}^n}$ is the *Fourier spectrum* of ϕ. A function with $k \ll 2^n$ non-zero Fourier coefficients is called Fourier-k-sparse, a property enforced over learned functions in Caro et al. [1]. Parities χ_s are the canonical example: they have exactly one non-zero Fourier coefficient $\hat{\chi}_s(s) = 1$.

Fourier analysis directly governs the misclassification error of parity-based hypotheses. Let $\mathcal{D} = (\mathcal{U}_n, \varphi)$ be a distribution over $\{0,1\}^n \times \{0,1\}$ with uniform input marginal, and let $\phi = 1 - 2\varphi$ denote the signed conditional label expectation. For any parity χ_s one can show:

$$\mathbb{P}_{(x,b) \sim \mathcal{D}}[b \neq \chi_s(x)] = \frac{1 - \hat{\phi}(s)}{2}$$

Consequently, the parity χ_{s^*} with *largest* Fourier coefficient $|\hat{\phi}(s^*)|$ minimises the misclassification error over all parities. Agnostic parity learning is therefore equivalent to identifying the heaviest Fourier coefficient of ϕ, which is exploited by the Goldreich-Leven algorithm [8] and central to the verification framework of Caro et al. [1].

More generally, for Fourier-k-sparse learning, the optimal k-sparse hypothesis under L^2 error measure is obtained by retaining the k-largest Fourier coefficients of ϕ.

2.4.2 Quantum Fourier Sampling for Parity

Under a distribution $\mathcal{D} = (\mathcal{U}_n, f), f : \{0,1\}^n \to \{0,1\}$ (the *functional-agnostic* setting in Section 2.3) returning a sample $(x, f(x))$ drawn uniformly at random, Equation 2 specialises to:

$$|\psi_{(\mathcal{U}_n,f)}\rangle = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} |x, f(x)\rangle.$$

The algorithm for *quantum Fourier sampling* [9] is remarkably simple here: apply the $(n+1)$-fold Hadamard $H^{\otimes(n+1)}$ to a single copy of $|\psi_{(\mathcal{U}_n,f)}\rangle$, then measure all $n+1$ qubits in the computational basis.

Conditioned on the last qubit measuring to 1 (which occurs with probability $\frac{1}{2}$ [1]), the first n qubits are observed in state $s \in \{0,1\}^n$ with probability (if measured in the computational basis):

$$\mathbb{P}[\text{outcome } s] = (\hat{g}(s))^2 \text{ with } g = (-1)^f$$

Proof. See [1, p. 19]. □

Repeated quantum Fourier sampling, combined with the Dvoretzky-Kiefer-Wolfowitz (DKW) inequality [10] allows efficient recovery of a succinct ℓ^∞-approximation to the entire Fourier spectrum of g.

However, classically, no efficient procedure is known for recovering the Fourier spectrum in the *distributional-agnostic* setting (Section 2.3) where labelling may be stochastic. Caro et al. attempt to resolve this by introducing *mixture-of-superpositions* examples, explored in Section 3.1.

2.5 Classically Verifying Quantum Learning

Following from the notion of an interactive proof system (Definition 5) and developing from the previous Section 2.4, the central question (which we implement in this report) becomes: can a classical verifier, with access to only classical random examples or statistical queries, reliably delegate an agnostic learning task to an untrusted quantum prover?

Goldwasser, Rothblum, Shafer, and Yehudayofff [11] introduce a formal framework of interactive proofs for Probably Approximately Correct (PAC) learning. A key observation of their work is that verification is trivial in the realizable PAC setting (the verifier can simply test the prover's hypothesis on fresh examples) but becomes non-trivial in the agnostic setting, where optimal achievable error is unknown to both parties. In the agnostic case, checking whether a candidate hypothesis is near-optimal requires knowledge of the best-in-benchmark-class error, which is what makes the learning problem hard.

However, Caro, Hinsche, Ioannou, Nieter, and Sweke [1] extend this framework to the quantum setting. Their main result establishes an efficient one-round interactive protocol to solve distributional 1-agnostic parity learning over a restricted class of distributions. We will explore this protocol in Section 3.

3 Mixture-of-Superpositions-based Verification Protocol

The goal of Caro, Hinsche, Ioannou, Nieter, and Sweke [1] was to discover a way of recovering the Fourier spectrum of a Boolean parity function in the distributional-agnostic setting, and verify its heaviest coefficients with only a classical verifier. They acieved this using their novel Mixture-of-Superpositions technique.

3.1 Mixture-of-Superpositions Construction

Let $\mathcal{D}$ be a distribution over $\mathcal{X}_n \times \{0,1\}$. Let $F_{\mathcal{D}}$ be the probability distribution over $\{0,1\}^{\mathcal{X}_n}$ defined by sampling $f(x)$ from the conditional label distribution independently for each $x \in \mathcal{X}_n$. Let $\varphi : \mathcal{X}_n \to [0,1] = \mathbb{E}_{(x,y)\sim\mathcal{D}}[y \mid x = z]$, the conditional label expectation of distribution $\mathcal{D}$. That is, for any $\tilde{f} : \mathcal{X}_n \to \{0,1\}$:

$$\begin{aligned}\mathbb{P}_{f\sim F_{\mathcal{D}}}\left[f = \tilde{f}\right] &= \prod_{z\in\mathcal{X}_n} \mathbb{P}_{(x,y)\sim\mathcal{D}}\left[\tilde{f}(z) = y \mid x = z\right] \\ &= \prod_{z\in\mathcal{X}_n} \Big(\varphi(z)\tilde{f}(z) + (1-\varphi(z))\big(1-\tilde{f}(z)\big)\Big)\end{aligned}$$

In simpler words, we say that the probability that the random f equals a specific $\tilde{f}$ is the product over all inputs of the probability that the label at z equals the label at $\tilde{f}(z)$.

> **Definition 7 (Mixture-of-superpositions quantum examples for distributional-agnostic learning)**. Let $\phi = 1 - 2\varphi$. Then a mixture-of-superpositions quantum example for $\mathcal{D}$ is a copy of the $(n+1)$-qubit state:
>
> $$\rho_{\mathcal{D}} = \mathbb{E}_{f\sim F_{\mathcal{D}}}\left[|\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}\rangle\langle\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}|\right]$$

We can see this follows from the fact that if there were a fixed deterministic target function $f : \mathcal{X}_n \to \{0,1\}$, then a quantum example consistent with marginal $\mathcal{D}_{\mathcal{X}_n}$ is the pure state:

$$|\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}\rangle = \sum_{x\in\mathcal{X}_n} \sqrt{\mathcal{D}_{\mathcal{X}_n}(x)}\ |x\rangle|f(x)\rangle \tag{3}$$

However, under a general distribution $\mathcal{D} = \mathcal{X}_n \times \{0,1\}$, there is no fixed target function and labels $\varphi(x) = y$ are random according to $\mathbb{P}[y = 1 \mid x]$. Therefore, there is no single pure state of the form in Equation 3 that correctly represents the data-generating process.

The distribution $F_{\mathcal{D}}$ allows us to overcome this as we sample a deterministic $\tilde{f} \sim F_{\mathcal{D}}$. Conditioned on $f = \tilde{f}$, labels are perfectly consistent and the quantum example is again pure. This defines the classical-quantum ensemble:

$$\left\{\mathbb{P}\left[f = \tilde{f}\right], |\phi_{(\mathcal{D}_{\mathcal{X}_n},\tilde{f})}\rangle\right\}_{\tilde{f}}$$

Since the learner does not observe f as a classically random variable, the unobserved classical randomness becomes mixing and creates a classical-quantum state on the joint space:

$$\rho_{cq} = \sum_f \mathbb{P}_{f\sim F_{\mathcal{D}}}[f]\ |f\rangle\langle f| \otimes |\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}\rangle\langle\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}|$$

Where $|f\rangle\langle f|$ is the projector encoding the classical event that the sampled function is f, and $|\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}\rangle\langle\phi_{(\mathcal{D}_{\mathcal{X}_n},f)}|$ is the density operator corresponding to the quantum example conditioned on f, meeting Definition 7.

3.1.1 Approximate Quantum Fourier Sampling from MoS

A formal theorem for the circuit we implement in Appendix E.

Theorem 2 (Approximate fourier sampling over Mixture of Superpositions). *Let $\mathcal{D}$ be a probability distribution over $\mathcal{X}_n \times \{0,1\}$ with $\mathcal{D}_{\mathcal{X}_n} = \mathcal{U}_n$. Let $\rho_{\mathcal{D}} = \mathbb{E}_{f\sim F_{\mathcal{D}}}\left[|\varphi_{(\mathcal{U}_n,f)}\rangle\langle\varphi_{(\mathcal{U}_n,f)}|\right]$ be our MoS quantum example from Definition 7. Given a copy of $\rho_{\mathcal{D}}$, first apply the unitary $H^{\otimes(n+1)}$, then measure all $n+1$ qubits in the computational basis. The measurement outcomes of this procedure satisfy the following:*

1. *The computational basis on the last qubit gives outcomes $\{0,1\}$ with equal probability.*
2. *Conditioned on having ovserved outcome 1 for the last qubit, the computational basis measurement on the first n qubits outputs a string $s \in \{0,1\}^n$ with probability*

$$\frac{1}{2^n}\left(1 - \mathbb{E}_{x\sim\mathcal{U}_n}\left[(\phi(x))^2\right]\right) + \left(\hat{\phi}(s)\right)^2 \tag{4}$$

Caro et al. [1] state that, with probability $\frac{1}{2}$, produce a sample from a distribution uniformly close to the squared Fourier spectrum of the label-bias function φ.

Proof. See [1, p. 26]. □

3.2 The Four-Step Protocol

Definition 8 (Distributions with L_2-bounded bias). Let $0 \le a \le b \le 1$. We denote the class of probability distributions $\mathcal{D} = (\mathcal{U}_n, \varphi)$ over $\mathcal{X}_n \times \{0,1\}$ that have a uniform marginal over $\mathcal{X}_n$, and whose $\{-1,1\}$-label expectation ϕ has squared L_2 norm in $[a^2, b^2]$ by:

$$\mathfrak{D}_{\mathcal{U}_n;[a^2,b^2]} := \left\{(\mathcal{U}_n, \varphi) : \mathbb{E}_{x\sim\mathcal{U}_n}\left[(\phi(x))^2\right] \in [a^2, b^2]\right\}$$

The motivation behind Definition 8 is derived from the requirement of the verifier checking whether the prover has provided a list with sufficient accumulated Fourier weight. Without such a promise, the total Fourier weight in the distributional-agnostic case may take any value $\in [0,1]$. With this established, we now state the distributional-agnostic version of classical verification for quantum parity learning.

Theorem 3 (Classical verification of quantum learning with MoS). *Let $\vartheta \in \left(2^{-(\frac{n}{2}-3)}, 1\right)$. Let $0 \leq a \leq b \leq 1$. Let $\delta \in (0,1)$ and $\varepsilon \geq 2\sqrt{b^2 - a^2}$. The class of n-bit parities is efficiently proper 1-agnostic verifiable w.r.t. $\mathcal{D}_{\mathcal{U}_n;\geq\vartheta} \cap \mathcal{D}_{\mathcal{U}_n;[a^2,b^2]}$ by a classical verifier V with access to classical random examples, interacting with a quantum prover P with access to MoS quantum examples. This can be achieved by a pair (V, P) that uses only a single round of communication. Caro et al. [1, p. 45] describe the following protocol for a classical V and honest quantum prover P:*

1. *V asks P to provide a list $L = \left\{s_1, ..., s_{|L|}\right\} \subset \{0,1\}^n$ of length $|L| \leq \frac{64b^2}{\vartheta^2}$ consisting of pairwise distinct n-bit strings whose associated Fourier coefficients are nonzero.*
2. *P follows the procedure in [1, p. 27] to produce, with success probability $\geq 1 - \frac{\delta}{2}$ a succinctly represented $\tilde{\phi} : \mathcal{X}_n \to [-1,1]$ such that $\|\tilde{\phi} - \hat{\phi}\|_\infty \leq \frac{\vartheta}{2}$ and $\|\tilde{\theta}\|_0 \leq \frac{64b^2}{\vartheta^2}$. If P obtains an output that violates the $\|\cdot\|_0$ bound, then P declares failure and the interation aborts. Otherwise, P sends the list $L = \left\{s \in \{0,1\}^n : |\tilde{\phi}(s) \geq \frac{\vartheta}{2}\right\}$ to V.*
3. *If V recieves a list L of length $|L| > \frac{64b^2}{\vartheta^2}$, V rejects the interaction. Otherwise, V uses $\mathcal{O}\left(\frac{|L|^2 \log\left(\frac{|L|}{\delta}\right)}{\varepsilon^4}\right)$ classical random examples from $\mathcal{D}$ to obtain simultaneously $\left(\frac{\varepsilon^2}{16|L|}\right)$-accurate estimates $\hat{\xi}(s)$ of $\hat{\phi}(s)$ for all $s \in L$ with success probability $\geq 1 - \frac{\delta}{2}$. For $t \notin L$, the verifier's estimate $\hat{\gamma}(t)$ for $\hat{g}(t)$ is just 0.*
4. *If $\sum_{\ell=1}^{|L|} \left(\hat{\xi}(s_\ell)\right)^2 \geq a^2 - \frac{\varepsilon^2}{8}$, then V determines $s_{\text{out}} \in \operatorname{argmax}_{1\leq\ell\leq|L|}\hat{\xi})s$ and outputs the hypothesis $h : \mathcal{X}_n \to \{0,1\}, h(x) = s_{\text{out}} \cdot x$. If $\sum_{\ell=1}^{|L|} \left(\hat{\xi}(s_\ell)\right)^2 < a^2 - \frac{\varepsilon^2}{8}$ then V outputs reject.*

Proof. See [1, p. 45]. □

3.3 Protocol Parameters (INCOMPLETE, CHECK)

The protocol's behaviour is governed by five interrelated parameters: the agnostic accuracy ε, the confidence δ, the Fourier-resolution threshold ϑ, and the bracket $[a^2, b^2]$ on $\mathbb{E}_{x\sim\mathcal{U}_n}\left[\tilde{\phi}(x)^2\right]$. Table 1 maps each symbol in [1, Theorem 12, p. 45] to the corresponding identifier in `mos/`, `ql/prover.py`, and `ql/verifier.py`.

The interlocking relationships from Theorem 12 are reproduced verbatim by the simulation. The accuracy is coupled to the bracket width through $\varepsilon \geq 2\sqrt{b^2 - a^2}$, which collapses to the unconstrained case $\varepsilon > 0$ when $a = b$ (e.g. the noiseless functional setting). The honest `MoSProver` extracts at most $16/\vartheta^2$ heavy frequencies — the tighter Parseval bound it can prove for itself — while `MoSVerifier._verify_core` enforces the looser malicious-prover bound $|L| \leq 64b^2/\vartheta^2$ on whatever list it receives. From the Hoeffding inequality, the verifier draws $\mathcal{O}\left(\frac{|L|^2 \log\left(|L\frac{|}{\delta}\right)}{\varepsilon^4}\right)$ classical examples to obtain simultaneous $\varepsilon^2/(16|L|)$-accurate estimates $\hat{\xi}(s)$, then accepts iff $\sum_{s\in L} \hat{\xi}(s)^2 \geq a^2 - \varepsilon^2/8$. Under the noisy distributional setting (`MoSState.noise_rate` $\eta > 0$), the effective spectrum is damped by $1 - 2\eta$, so the experimental harness sets $a^2 = b^2 = (1 - 2\eta)^2$ for the parity case.

Symbol	**Caro et al. (Thm 12, p. 45)**	**Codebase identifier**	**Module**
n	number of input bits	`MoSState.n`	`mos`
η	label-flip noise rate	`MoSState.noise_rate`	`mos`
$\tilde{\phi}$	$1-2\varphi$, the ± 1-valued bias	`MoSState.tilde_phi`	`mos`
$\tilde{\phi}_{\text{eff}}$	$(1-2\eta)\tilde{\phi}$	`MoSState.tilde_phi_effective`	`mos`
$\rho_{\mathcal{D}}$	MoS state	`MoSState` + `QuantumFourierSampler`	`mos`
ϑ	Fourier resolution threshold	`theta` kwarg	`ql/{prover,verifier}`
ε	agnostic accuracy ($\geq 2\sqrt{b^2-a^2}$)	`epsilon` kwarg	`ql/{prover,verifier}`
δ	confidence parameter	`delta` kwarg	`ql/{prover,verifier}`
a^2, b^2	bracket on $\mathbb{E}[\tilde{\phi}^2]$ (Def. 14)	`a_sq`, `b_sq` (`verify_parity`)	`ql/verifier`
L	heavy-coefficient list	`ProverMessage.L`	`ql/prover`
$64b^2/\vartheta^2$	list-size bound (Step 3)	`list_size_bound`	`ql/verifier`
Step 1 copies	QFS shots, $\mathcal{O}\big(\log\big(\frac{1}{\delta}\big)/\vartheta^4\big)$	`qfs_shots` (auto from δ, ϑ)	`ql/prover`
Step 2 examples	$\mathcal{O}\Big(\Big(\lvert L\rvert^2 \log\big(\lvert L\rvert\frac{1}{\delta}\big)\Big)/\varepsilon^4\Big)$	`num_classical_samples`	`ql/verifier`
$\varepsilon^2/(16\lvert L\rvert)$	per-coefficient tolerance	`per_coeff_tolerance`	`ql/verifier`
$a^2-\varepsilon^2/8$	acceptance threshold (Step 4)	`acceptance_threshold`	`ql/verifier`

Table 1: Mapping of Theorem 12 parameters to codebase identifiers. Sample-complexity rows give the auto-computed defaults used when no explicit override is passed; both `MoSProver.run_protocol` and `MoSVerifier.verify_parity` accept manual overrides for ablation experiments.

Notes on a few subtleties baked into the prose so you can decide whether to keep them:

- Honest-prover vs.
  malicious-prover list bound. ql/prover.py:490 truncates L at 16 / theta2 (the tight Parseval bound), while ql/verifier.py:464 allows up to ceil(64 times b_sq / theta^2). The discrepancy is intentional and matches Caro et al. — the verifier needs the looser bound to be sound against an adversary that pads L.
- Auto-computed sample budgets. MoSProver.run_protocol derives qfs_shots from 2.5 **2** log(4/δ) / (ϑ²/8)²

(ql/prover.py:316-321) and MoSVerifier._verify_core derives num_samples from the Hoeffding bound (ql/verifier.py:493-501); both expose overrides which is why I phrased the table rows as defaults.

- Noise feeding into a², b². The dissertation harness translates noise_rate η into a_sq = b_sq = (1

- $2\eta)^2$. If your

experiments don't all do that — e.g., the gate-noise sweep may use a different convention — let me know and I'll adjust the last sentence.

TODO: FINISH DRAFTING THIS

4 Implementation of the Mixture-of-Superpositions Verification Protocol

4.1 Architecture Overview

The implementation of the Caro et al. [1] protocol in Theorem 3 is split into two Python packages that reflect a deliberate separation between *state*-level quantum machinery and *protocol*-level interactive verification. Throughout this implentation, we heavily rely on the Qiskit [12] package, an open source, Python-based, high-performance software stack for quantum computing.

Package	Module	Public classes	Report anchor
`mos`	`mos/__init__.py`	`MoSState`	Definition 7 (MoS state); Section 3.1
`mos`	`mos/sampler.py`	`QuantumFourierSampler`; `QFSResult`	Section 3.1.1
`ql`	`ql/prover.py`	`MoSProver`; `SpectrumApproximation`; `ProverMessage`	Theorem 3; Corollary 5 and Lemma 3 of Caro et al. [1]
`ql`	`ql/verifier.py`	`MoSVerifier`; `ParityHypothesis`; `FourierSparseHypothesis`; `VerificationResult`	Theorem 3

Table 2: A summary of files directly responsible for protocol simulation and their corresponding links to theory.

4.2 Mixture-of-Superpositions Simulator

The `mos/__init__.py` module (Appendix A) provides a faithful implementation of the mixture-of-superpositions-based quantum example `MoSState` from Definition 7. All other modules treat the class `MoSState` as a black-box of either copies of the mixed quantum example state $\rho_{\mathcal{D}}$, or classical samples $(x, y) \sim \mathcal{D}$. Note that we use Lemma 1 from [1] to draw equivalence between computational basis measurements on all $n + 1$ qubits of $\rho_{\mathcal{D}}$ and producing a sample from $\mathcal{D}$ itself.

To build this state, we require the function distribution $F_{\mathcal{D}}$ over $\{0, 1\}^{\mathcal{X}_n}$ defined by sampling $f(x) : \mathcal{X}_n \to \{0, 1\}$ from the conditional label distribution independently for each x. We achieve this by storing φ as a length-2^n array (an input for the constructor of `MoSState`), with an input noise rate η. We then pre-compute φ_{eff}, the effective phi $\mathbb{P}[y = 1 \mid x]$ after independent label flips from noise.

```
# mos/__init__.py:127-129
eta = self.noise_rate
self._phi_effective: np.ndarray = (1 - 2 * eta) * self._phi + eta
```

Listing 1: Precomputing φ_{eff}

To simulate sampling $f \sim F_{\mathcal{D}}$, we can compute 2^n independent Bernoulli draws against φ_{eff}.

```
# mos/__init__.py:185-187
def sample_f(self, rng: Optional[Generator] = None) -> np.ndarray:
  if rng is None:
      rng = self._rng
  return (rng.random(self.dim_x) < self._phi_effective).astype(np.uint8)
```

Listing 2: Vectorized $f \sim F_{\mathcal{D}}$ sampling.

Notice that by $\mathbb{P}_{f\sim F_{\mathcal{D}}}\left[f = \tilde{f}\right]$ from Section 3.1, the product form is exactly what makes labels independent across inputs. Function values at each x depend only on $\varphi(x)$, producing a binary array where:

$$f(x) = \begin{cases} 1 \text{ with probability } \varphi_{\text{eff}}(x) \\ 0 \text{ with probability } 1 - \varphi_{\text{eff}}(x) \end{cases}$$

Crucially, the rest of the protocol does not see f directly, it will either work with state $\rho_{\mathcal{D}}$ or marginal $\mathcal{D}$, of which both are mixtures over f.

Take f to now be fixed. We can perform the standard textbook preparation of $|\psi_{(\mathcal{U}_n,f)}\rangle = \frac{1}{\sqrt{2^n}}\sum_x |x, f(x)\rangle$ by applying $H^{\otimes n}$ on the x register, followed by an oracle that flips the label qubit on every x where $f(x) = 1$:

```
# mos/__init__.py:297-308
qr = QuantumRegister(self.n + 1, "q")
qc = QuantumCircuit(qr, name="prepare_psi_f")

for i in range(self.n):
    qc.h(qr[i])

oracle = self._circuit_oracle_f(f)
qc.compose(oracle, inplace=True)

return qc
```

Listing 3: Circuit construction for f

Following this, we define the oracle flipping the label qubit on $x : f(x) = 1$ `_circuit_oracle_f`:

```python
# mos/__init__.py:251-269
qr = QuantumRegister(self.n + 1, "q")
qc = QuantumCircuit(qr, name="oracle_f")

for x in range(self.dim_x):
    if f[x] == 1:
        ctrl_state = format(x, f"0{self.n}b")
        if self.n == 1:
            qc.cx(0, 1, ctrl_state=ctrl_state)
        else:
            qc.mcx(
                control_qubits=list(range(self.n)),
                target_qubit=self.n,
                ctrl_state=ctrl_state,
            )

return qc
```

Listing 4: Circuit oracle f.

We note a practical limitation here: the construction is brute-force, creating up to 2^n multi-controlled gates, one for each $x : f(x) = 1$. This is feasible for small simulations, however impractical for large n. Instead, we propose an alternative `Statevector`-based approach.

For a fixed f, the state $|\psi_{(\mathcal{U}_n,f)}\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x, f(x)\rangle$ has exactly 2^n nonzero amplitudes, all equal to $2^{\frac{-n}{2}}$. We write these using Qiskit's [12] little-endian convention. Qubit n is the highest-index qubit and so maps $|x, b\rangle$ to index $x + b \cdot 2^n$.

```python
# mos/__init__.py:217-224
sv_data = np.zeros(self.dim_total, dtype=np.complex128)
amp = 1.0 / np.sqrt(self.dim_x)

for x in range(self.dim_x):
    idx = x + int(f[x]) * self.dim_x
    sv_data[idx] = amp

return Statevector(sv_data)
```

Listing 5: Statevector for f construction.

Lemma 1. The `statevector`-based construction of f and the true-to-literature circuit based construction of f are identical under the assumption of no gate noise; both produce, for any $f : \{0,1\}^n \to \{0,1\}$, the state:

$$|\psi_f\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x, f(x)\rangle$$

Proof. We write $|x, b\rangle$ for the basis state where the input register holds $x \in \{0, ..., 2^n - 1\}$ and the label qubit holds $b \in \{0, 1\}$. Under the little-endian convention, this corresponds to array index $x + b \cdot 2^n$.

The method `statevector_f` constructs a vector $v \in \mathbb{C}^{2^{n+1}}$ defined by:

$$v_j = \begin{cases} \frac{1}{\sqrt{2^n}} \text{ if } j = x + f(x) \cdot 2^n \text{ for some } x \in \{0, ..., 2^n - 1\} \\ 0 \text{ otherwise} \end{cases}$$

Since the map $x \to x + f(x) \cdot 2^n$ is injective, exactly 2^n entries are nonzero, each equal to $\frac{1}{\sqrt{2^n}}$. The squared norm is $2^n \cdot \frac{1}{2^n} = 1$. By the index convention, v represents:

$$\frac{1}{\sqrt{2^n}} \sum_x |x, f(x)\rangle = |\psi_f\rangle$$

All other basis states, namely $|x, \overline{f(x)}\rangle$ recieve zero amplitude due to a deterministic f.

For the circuit-based preparation, we start with an application of $(H^{\otimes n} \otimes I)$ to input register $|0\rangle^{\otimes(n+1)}$:

$$(H^{\otimes n} \otimes I)|0\rangle^{\otimes(n+1)} = \frac{1}{\sqrt{2^n}} \sum_x |x, 0\rangle$$

Then, it suffices to show the oracle defined on computational basis states by the unitary $U_f|x, b\rangle = |x, b \oplus f(x)\rangle$ that:

$$U_f \cdot \frac{1}{\sqrt{2^n}} \sum_x |x, 0\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x, f(x)\rangle = |\psi_f\rangle$$

For each $x : f(x) = 1$, the circuit places an *MCX* gate that flips qubit n conditioned on the input register being $|x\rangle$. Gates for distinct x, x' commute because they have orthogonal control states, so at most one fires on any basis state. Since the circuit contains only gates for $x : f(x) = 1$, the label qubit is flipped if and only if $f(x) = 1$. That is, the circuit maps $|y, b\rangle \to |y, b \oplus f(y)\rangle$, which is exactly U_f. □

For classical sampling of $f \sim F_{\mathcal{D}}$, we can simulate this by measuring the mixed state $\rho_{\mathcal{D}}$ in the computational basis [1].

Proof. By definition of $\rho_{\mathcal{D}}$, the probability of observing output string $(x, b) \in \mathcal{X}_n \times \{0, 1\}$ when measuring all $(n + 1)$ qubits in the computational basis is given by:

$$\begin{aligned} \langle x, b|\rho_{\mathcal{D}}|x, b\rangle &= \mathbb{E}_{f \sim F_{\mathcal{D}}}\left[|\langle x, b| \ \psi_{(\mathcal{D}_{\mathcal{X}_n}, f)}\rangle|^2\right] \\ &= \mathbb{E}_{f \sim F_{\mathcal{D}}}\left[\mathcal{D}_{\mathcal{X}_n}(x)\delta_{b, f(x)}\right] \\ &= \mathcal{D}_{\mathcal{X}_n}(x)\mathbb{P}_{f \sim F_{\mathcal{D}}}[f(x) = b] \\ &= \mathcal{D}(x, b) \end{aligned}$$

As claimed. □

To avoid overhead of quantum simulation, we can instead sample $x \sim U_n$, and $y \sim \text{Bernoulli}(\varphi_{\text{eff}}(x))$ directly, which works on the same distribution, but without having to use a $2^{(n+1)}$-dimension state vector.

```
# mos/__init__.py:412-417
if rng is None:
  rng = self._rng # a numpy.Generator

x = rng.integers(0, self.dim_x)
y = int(rng.random() < self._phi_effective[x])
return x, y
```

Listing 6: A singular sample $x \sim U_n$ and $y \sim \text{Bernoulli}(\varphi_{\text{eff}}(x))$.

We provide a vectorized implementation `sample_classical_batch`, available in Appendix A.6.

4.2.1 Implementation of Fourier analysis over MoS

Recall from Definition 6, for the Fourier coeffient $\hat{\phi}(s)$ where $\chi_s(x) = (-1)^{s \cdot x}$ and $\phi = 1 - 2\varphi$:

$$\hat{\phi}(s) = \mathbb{E}_{x \sim U_n}[\phi(x) \cdot \chi_s(x)]$$

Then, we can calculate the exact Fourier coefficient at s with:

```
# mos/__init__.py:482-489
tphi = self.tilde_phi
parities = np.array([bin(s & x).count("1") % 2 for x in range(self.dim_x)])
chi_s = 1.0 - 2.0 * parities  # (-1)^{s·x}
coeff = float(np.mean(tphi * chi_s))
if effective:
    coeff *= self._noise_damping
return coeff
```

Listing 7: The fourier coefficient calculation for some specific s.

We provide both a function to compute the full Fourier spectrum `fourier_spectrum` for all s via repetition of the above, as well as a function to verify Parseval's identity `parseval_check`:

$$\sum_s \hat{\phi}(s)^2 = \mathbb{E}[\phi(x)^2],$$

available in Appendix A.7.

Now recall from Equation 4 that if one applies $H^{\otimes(n+1)}$ to a copy of $\rho_{\mathcal{D}}$, and measures all qubits in the computational basis post-selecting on the label qubit being $|1\rangle$, then the conditional distribution over the first n bits is:

$$\mathbb{P}[s \mid b = 1] = \frac{1}{2^n}\left(1 - \mathbb{E}_{x \sim \mathcal{U}_n}\left[(\phi(x))^2\right]\right) + \left(\hat{\phi}(s)\right)^2$$

We compute this as follows:

```
tphi_eff = self.tilde_phi_effective
E_sq = float(np.mean(tphi_eff**2))
coeff = self.fourier_coefficient(s, effective=True)
return (1.0 - E_sq) / self.dim_x + coeff**2
```

Listing 8: Computed probability of observing s, conditioned on the last qubit being 1.

We provide a function `qfs_distribution` to calculate the full QFS distribution over $\{0,1\}^n$ in Appendix A.7.

4.2.2 Sampling from `MoSState`

The package `mos/sampler.py` is the only place in the codebase of which performs Hadamard-basis measurements of a MoS state, depending only on `MoSState` from Appendix A. In the case of `Statevector`-based simulation, we sample an independent f per "shot", and draw a single measurement outcome per f:

```
# mos/sampler.py:312-332
n = self.n
dim_total = self.state.dim_total
counts: dict[str, int] = {}

h_circuit = QuantumCircuit(n + 1, name="H_all")
for q in range(n + 1):
    h_circuit.h(q)

for _ in range(shots):
    f = self.state.sample_f(rng=self._rng)
    psi_f = self.state.statevector_f(f)
    psi_h = psi_f.evolve(h_circuit)

    probs = psi_h.probabilities()
    idx = self._rng.choice(dim_total, p=probs)
    bitstring = format(idx, f"0{n + 1}b")
    counts[bitstring] = counts.get(bitstring, 0) + 1

return counts
```

Listing 9: Sampling f from the `MoSState` construction and performing a final measurement on the evolved state.

We provide a circuit-based sampler as well as the postselection mechanism in Appendix B.2.1, with dispatch between sampling mechanisms as follows:

```
# mos/sampler.py:227:241
dispatch = {
    "statevector": self._sample_statevector,
    "circuit": self._sample_circuit,
}
raw_counts = dispatch[mode](shots)
ps_counts, ps_shots = self._postselect(raw_counts)

return QFSResult(
    raw_counts=raw_counts,
    postselected_counts=ps_counts,
    total_shots=shots,
    postselected_shots=ps_shots,
    n=self.n,
    mode=mode,
)
```

Listing 10: `sample` function dispatch

Note that we store `postselected_counts` - although not a meaningful statistic for the actual verification procedure in Theorem 3, it is important for checking protocol correctness to theory. We collect a number of statistics in a similar fashion for testing.

4.3 Quantum Prover

Recall Theorem 3, step 2. The prover P is responsible for obtaining a succinctly represented $\tilde{\phi} : \mathcal{X}_n \to [-1, 1]$.

> **Corollary 1**. *Let $\mathcal{D}$ be a probability distribution over $\mathcal{X}_n \times \{0,1\}$ with $\mathcal{D}_{\mathcal{X}_n} = \mathcal{U}_n$. Let $\delta, \varepsilon \in (0,1)$. Assume $\varepsilon > 2^{-(\frac{n}{2}-2)}$. Then there exists a quantum algorithm that, given $\mathcal{O}\left(n\frac{\log\left(\frac{1}{\delta\varepsilon^2}\right)}{\varepsilon^4}\right)$ copies of $\rho_{\mathcal{D}}$, uses an efficient number of single-qubit gates, an efficient amount of classical memory and an efficient amount of classical computation time, outputs with success probability $\geq 1-\delta$, a succinctly represented $\tilde{\phi} : \mathcal{X}_n \to [-1,1]$ such that $\|\tilde{\phi} - \hat{\phi}\|_\infty \leq \varepsilon$ and $\|\tilde{\varphi}\|_0 \leq \frac{16\mathbb{E}_{x\sim\mathcal{U}_n}[(\phi(x)^2)]}{\varepsilon^2} \leq \frac{16}{\varepsilon^2}$.*

Proof. As from Caro et al. [1, p. 28], but restated informally for conciseness. We begin with Equation 4 after applying $H^{\otimes(n+1)}$ to a single copy of $\rho_{\mathcal{D}}$ and measuring. The first term of this statement is at most 2^{-n}, so this samples approximately from the squared Fourier spectrum.

Sample $\rho_{\mathcal{D}}$ a total of $m = \mathcal{O}\left(\frac{\log(\frac{1}{\delta})}{\varepsilon^4}\right)$ times. By the DKW inequality [10], the empirical distribution $\tilde{q}_m$ satisfies $\|q - \tilde{q}_m\| \leq \frac{\varepsilon^2}{8}$ with probability $\geq 1 - \frac{\delta}{2}$.

Compile the list $L = \left\{s : \tilde{q}_m(s,1) \geq \frac{\varepsilon^2}{4}\right\}$. The approximation guarantee ensures:

$$|\hat{\phi}(s)| \geq \varepsilon \Rightarrow s \in L (\text{completeness})$$

$$s \in L \Rightarrow |\hat{\phi}(s)| \geq \frac{\varepsilon}{4} (\text{soundness})$$

By Parseval's identity (Theorem 1) we can bound $|L| \leq \frac{16}{\varepsilon^2}$. □

We implement the prover `P`'s entire conversation with the verifier in one method, `run_protocol` (Appendix C.4). First computing the number of required shots m:

```
#ql/prover.py:315-320
tau = theta**2 / 8.0
if qfs_shots is None:
    m_postselected = int(np.ceil(2.0 * np.log(4.0 / delta) / tau**2))
    # Post-selection succeeds ~1/2 the time, so need ~2m total shots.
    # Add a safety margin for finite-sample fluctuation.
    qfs_shots = int(np.ceil(2.5 * m_postselected))
```

After running the sampler from Section 4.2.2, we convert the raw postselected QFS counts into the sparse `SpectrumApproximation` (Appendix C.1), and finally extract heavy coefficients in Appendix C.6. We also provide a diagnostic estimator for coefficients, as well as a function for exact heavy coefficients for testing purposes.

4.4 Classical Verifier

The honest quantum prover from Section 4.3 and the verifier V we implement here together realise the (V, P) pair of Theorem 3.

Upon reciept of prover P's list, we immediately perform the list size check in step three of Theorem 3, before any classical samples are drawn from $\mathcal{D}$:

```
# ql/verifier.py:464-479
list_size_bound = int(np.ceil(64.0 * b_sq / theta**2))

if len(L) > list_size_bound:
    return (
      VerificationResult(
        outcome=VerificationOutcome.REJECT_LIST_TOO_LARGE, ...
      )
    )
```

Listing 12: List size bound derived from Theorem 1.

Then, performing independent re-estimation, we calculate a per-coefficient tolerance depending on the hypothesis type from Appendix D.1, and derive the accumulated-weight check from Caro et al. [1, eq. 109, 118]:

```
# ql/verifier.py:486-533
if hypothesis_type == HypothesisType.PARITY:
    per_coeff_tolerance = epsilon**2 / (16.0 * max(L_size, 1))
else:
    per_coeff_tolerance = epsilon**2 / (256.0 * k**2 * max(L_size, 1))

num_samples = int(
    np.ceil(2.0 / per_coeff_tolerance**2 * np.log(4.0 * L_size / delta))
)

accumulated_weight = sum(verifier_estimates.get(s, 0.0) ** 2 for s in L)
if hypothesis_type == HypothesisType.PARITY:
    acceptance_threshold = a_sq - epsilon**2 / 8.0
else:
    acceptance_threshold = a_sq - epsilon**2 / (128.0 * k**2)

if accumulated_weight < acceptance_threshold:
    return VerificationResult(
        outcome=VerificationOutcome.REJECT_INSUFFICIENT_WEIGHT, ...
    )
```

Listing 13: Tolerance and acceptance-threshold bounding on results for verification.

The variable `verifier_estimates` from above is derived by a coefficient estimation function within the verifier completely independent of the prover.

```python
# ql/verifier.py:597-617
xs, ys = self.state.sample_classical_batch(
    num_samples=num_samples,
    rng=self._rng,
)

signed_labels = 1.0 - 2.0 * ys.astype(np.float64)
estimates: dict[int, float] = {}
for s in L:
    parities = np.array(
        [bin(s & int(x)).count("1") % 2 for x in xs],
        dtype=np.float64,
    )
    chi_s = 1.0 - 2.0 * parities
    est = float(np.mean(signed_labels * chi_s))
    estimates[s] = est
```

Listing 14: Classical sampling protocol for the verifier.

5 Experiments and Results

This chapter reports 7 experiments that evaluate the mixture-of-superpositions verification protocol of Caro et al. [1]. Each experiment tests a specific theorem, lemma, or corollary of the paper on a dedicated target class. All experiments were ran on the University of Warwick's DCS batch compute system, specifically on the `tiger` node, with experiments supporting distributed trials and Protobuf-based result aggregation and processing.

The four experimental areas are: completeness on honest provers (Section 5.1), soundness on dishonest provers (Section 5.2), robustness to noise and distributional assumptions (Section 5.3), and parameter sensitivity and practical limitations (Section 5.4). Three structural caveats apply throughout: firstly, every experiment hard-codes sample budgets at values that are orders of magnitude less than the analytic Hoeffding requirement of Caro et al. [1, Thm. 12], so the experiment maps boundaries and mechanisms rather than asymptotic complexity. This is an explicit limitation of simulating the computations on classical machinery. Next, some experiments deliberately operate *off-promise* in order to probe failure cases where formal preconditions are violated. Third, soundness in Caro et al. [1, Definition 7] is universally quantified. The cheating strategies in this chapter constitute an empirical spot-check as opposed to a proof of the universal statement.

Unless otherwise stated, let $\varepsilon = 0.3, \delta = 0.1, \vartheta = \varepsilon = 0.3$, and $a^2 = b^2 = 1$.

5.1 Completeness Experiments

The following experiments measure whether the verifier accepts the honest MoS prover and recovers a hypothesis with low error. We begin by defining what *completeness* and *soundness* mean in the context of our learning task.

Definition 9 (Interactive verification of α-agnostic learning). Let $\mathcal{F} \subseteq \{0,1\}^{\mathcal{X}_n}$ be a benchmark class. Let $\mathfrak{D}$ be a family of probability distributions over $\mathcal{X}_n \times \{0,1\}$. Let $\alpha \geq 1$. We say that $\mathcal{F}$ is α-agnostic verifiable with respect to $\mathfrak{D}$ if there exists a pair of classical or quantum algorithms (V, P) that satisfy the following conditions for every input accuracy parameter $\varepsilon \in (0,1)$ and every confidence parameter $\delta \in (0,1)$:

- **Completeness**: For any $\mathcal{D} \in \mathfrak{D}$ the random hypothesis $h : \mathcal{X}_n \to \{0,1\}$ that $V(\varepsilon, \delta)$ outputs after interacting with honest prover P satisfies:

$$\mathbb{P}[h \neq \text{reject} \wedge (\text{err}_{\mathcal{D}}(h) \leq \alpha \cdot \text{opt}_{\mathcal{D}}(\mathcal{F}) + \varepsilon)] \geq 1 - \delta \tag{5}$$

- **Soundness**: For any $\mathcal{D} \in \mathfrak{D}$ and for any dishonest prover P', the random hypothesis $h : \mathcal{X}_n \to \{0,1\}$ that $V(\varepsilon, \delta)$ outputs after interacting with P' satisfies:

$$\mathbb{P}[h \neq \text{reject} \wedge (\text{err}_{\mathcal{D}}(h) > \alpha \cdot \text{opt}_{\mathcal{D}}(\mathcal{F}) + \varepsilon)] < \delta \tag{6}$$

5.1.1 Scaling baseline

The scaling experiment `experiments/harness/scaling.py` is the pure completeness baseline for [1, Thm. 12]. Each trial draws a uniformly random non-zero parity $s^* \in \{1, ..., 2^n - 1\}$, producing a noiseless functional target $\varphi(x) = s^* \cdot x$ for which the Fourier spectrum is the indicator $\hat{\hat{\varphi}} = \chi_{s^*}$ with $a^2 = b^2 = 1$ (no noise).

The experiment is designed to confirm four predictions: that $\mathbb{P}[\text{accept}] \geq 1 - \delta$; the postselection rate is $\frac{1}{2}$; the list-size bound; the wall-clock cost of simulation on classical computers as n grows.

Data was collected into `results/scaling_4_16_100.pb`, with 1300 trials spanning $n \in \{4, ..., 16\}$ and 100 trials per n. Parameters were $\varepsilon = 0.3, \delta = 0.1, \vartheta = \varepsilon = 0.3$, and $a^2 = b^2 = 1$. Sample budgets were hard-coded, with a fixed total of 6000 copies of $\rho_{\mathcal{D}}$ per trial independent of n.

Acceptance and correctness of acceptance were 100% in all cases, so well above the required $1 - \delta = 0.9$. Median post-selection rate lay in $[0.496, 0.502]$ across all n, matching Theorem 2′s $\frac{1}{2}$-post-selection rate.

Wall-clock time rose from 0.36s at $n = 4$ to 1203s at $n = 16$, and is entirely dominated by the 2^{n+1}-dimensional statevector simulator and not by any protocol quantity, seen in Figure 1.

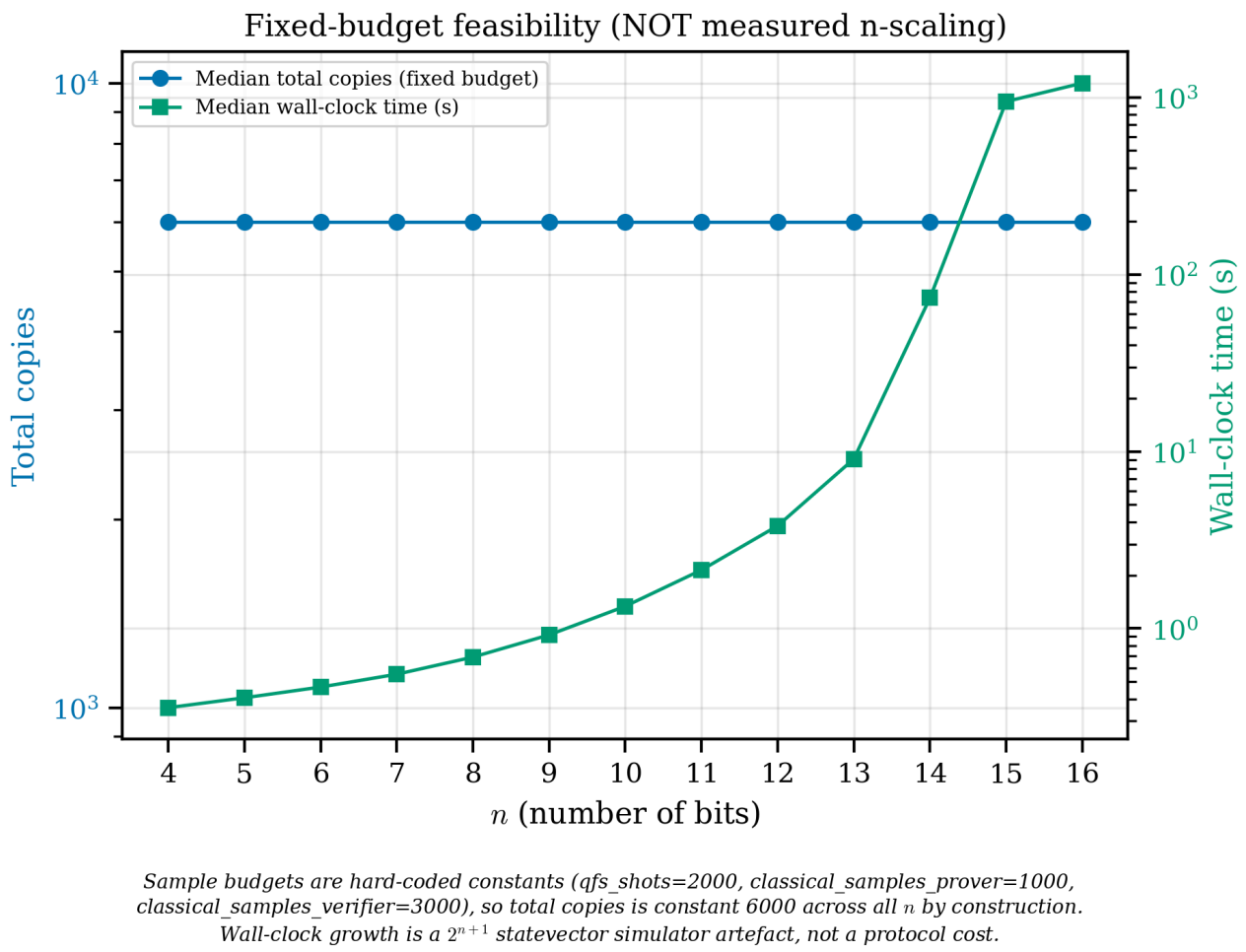

Figure 1: Fixed-budget feasibility. Total samples per trial are pinned at (6,000); the curve shows wall-clock time, whose growth is dominated by the 2^{n+1}-dimensional statevector simulator in `mos/__init__.py`, not by any protocol-level cost.

5.2 Soundness Experiments

The soundness experiments measure whether the verifier rejects dishonest provers, instantiated as a finite menu of hand-written cheating strategies that bypass the prover and feed adversarial input directly to the verifier. Caro et al. [1, Definition 7] state that the probability that the verifier accepts a hypothesis with error exceeding $\alpha \cdot \mathrm{opt} + \varepsilon$ is at most δ.

Section 5.2.1 treats single-parity targets and four cheating strategies; Section 5.2.2 treats k-sparse targets and a separate menu of strategies that probe the Caro et al. [1, Thm. 15] weight check.

5.2.1 Single-parity dishonest provers

Definition 9 requires that against any dishonest P', the verifier output h satisfies Equation 6. The harness file `experiments/harness/soundness.py` bypasses the honest prover entirely and feeds four adversarial transcripts directly into the verifier, recording whether V accepts and whether the resulting hypothesis is ε-close to the planted parity.

The data set `results/soundness_4_20_100.pb` contain 6800 trials across $n \in \{4, ..., 20\}$, 100 trials per n. Thorughout, we fix $\varepsilon = 0.3, \delta = 0.1, \vartheta = \varepsilon = 0.3, a^2 = b^2 = 1$ and $m_V = 3000$

verifier samples. The four cheating strategies are: a *random list*, five distinct random indicies (so each s^* is present with probability $\frac{5}{2^n}$); *wrong parities*, the singleton $\{(s^* + 1) \bmod 2^n\}$; the empty list $L = []$; and an *inflated list* with ten random indices excluding s^*.

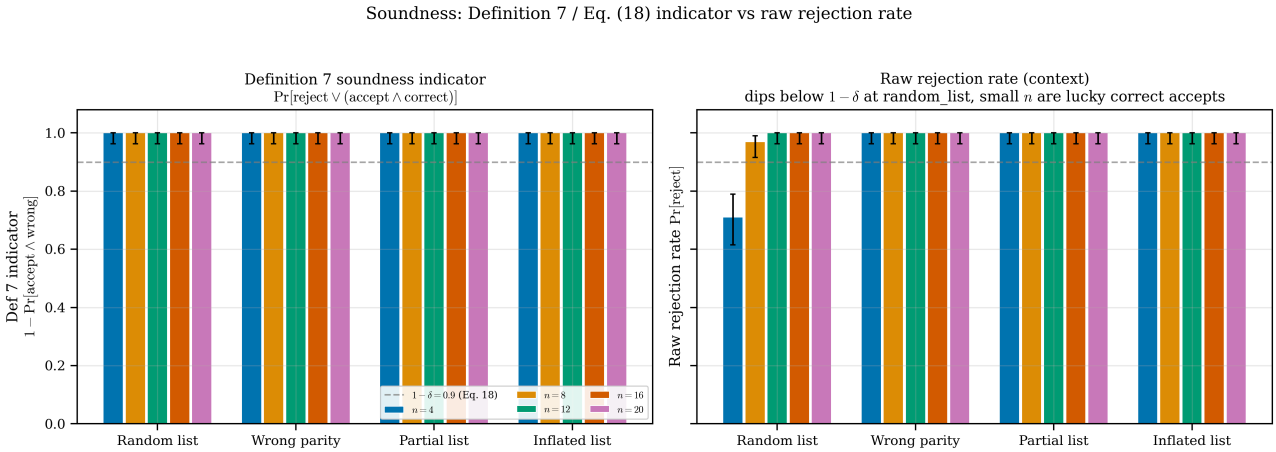

Figure 2: Soundness against the four hand-written cheating strategies. On the left is the indicator $1 - \mathbb{P}[\text{accept} \wedge \text{wrong}]$. On the right, the raw rejection rate.

Strategy	$n = 4$	$n = 8$	$n = 12$	$n = 16$	$n = 20$	Bad accept
Random list	71%	97%	100%	100%	100%	0/1700
Wrong parity	100%	100%	100%	100%	100%	0/1700
Empty list	100%	100%	100%	100%	100%	0/1700
Inflated list	100%	100%	100%	100%	100%	0/1700

Table 3: Per-strategy rejection rate.

We distinguish acceptance in incorrect cases from a soundness violation: a cheating strategy may happen to submit a hypothesis that is in fact ε-good, in which case acceptance is correct. This is visible in Figure 2, Table 3, where the 5-element list in the *random list* case happens to contain s^*. The observed counts match the analytical $\frac{5}{2^n}$ prediction within a safe interval; at $n = 4$, we predict 31 accepts out of 100 and observe 29.

In all rejection cases, the weight-check of Theorem 3 is responsible for rejection in every cell. This is due to the implemented list-size bound $\frac{64b^2}{\vartheta^2}$ never being triggered, because the largest adversarial list submitted has $|L| = 10$. This is a weakness of the experiment and an extension of future work to test this specific failure case.

5.2.2 Multi-element dishonest provers

In this experiment, we sweep a cheating prover through four multi-element dishonest strategies: *partial real* being the weaker half of the real set of heavy coefficients and 3 random fakes; *diluted list*, $\frac{k}{2}$ weakest real coefficients and up to 20 padding; *entirely wrong* indices with fabricated large coefficients; and *subset plus noise*, which is the single heaviest Fourier coefficient with 5 fakes.

Strategy	$k = 2$ mean	$k = 2$ min	$k = 4$ mean	$k = 4$ min
Partial real	100%	100%	100%	100%
Diluted list	98.5%	96%	100%	100%
Entirely wrong	100%	100%	100%	100%
Subset plus noise	95.1%	91%	99.8%	98%

Table 4: Rejection rates by strategy and Fourier-sparsity k.

The most significant boundary cell in Table 4 is for the strategy submitting the single-heaviest real Fourier coefficient plus five random fakes, so it carries real signal.

5.3 Robustness Experiments

The robustness experiments probe how the protocol degrades when one of its assumptions is perturbed. In Section 5.3.1, we sweep the η noise parameter from Caro et al. [1, Lemma 6], and in Section 5.3.2, we report a purely exploratory study of per-gate depolarising noise (of which Caro et al. make no theoretical prediction).

5.3.1 Label-flip noise sweep

For a target parity s^* [1, Lemma 6] gives the conditional QFS distribution:

$$\mathbb{P}[s|b = 1] = \frac{4\eta - 4\eta^2}{2^n} + (1 - 2\eta)^2 \hat{g}(s)^2, g = (-1)^f$$

So the fourier mass accumulated at s^* should track $(1 - 2\eta)^2$ exactly, perturbed only by an n -independent uniform term. We observe the protocol entering its formal breakdown at $\eta_{\max} \approx 0.4470$, the root at which the $(1 - 2\eta)^2 - \frac{\varepsilon^2}{8}$ of Theorem 3 becomes negative.

The noise sweep experiment contains ≈ 17000 trials across $n \in \{4, ..., 16\}$ and 13 noise rates

$$\eta \in \{0.00, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.42, 0.44, 0.46, 0.48\}$$

Where the four largest η cross the aforementioned theoretical breakdown for $\varepsilon = 0.3$.

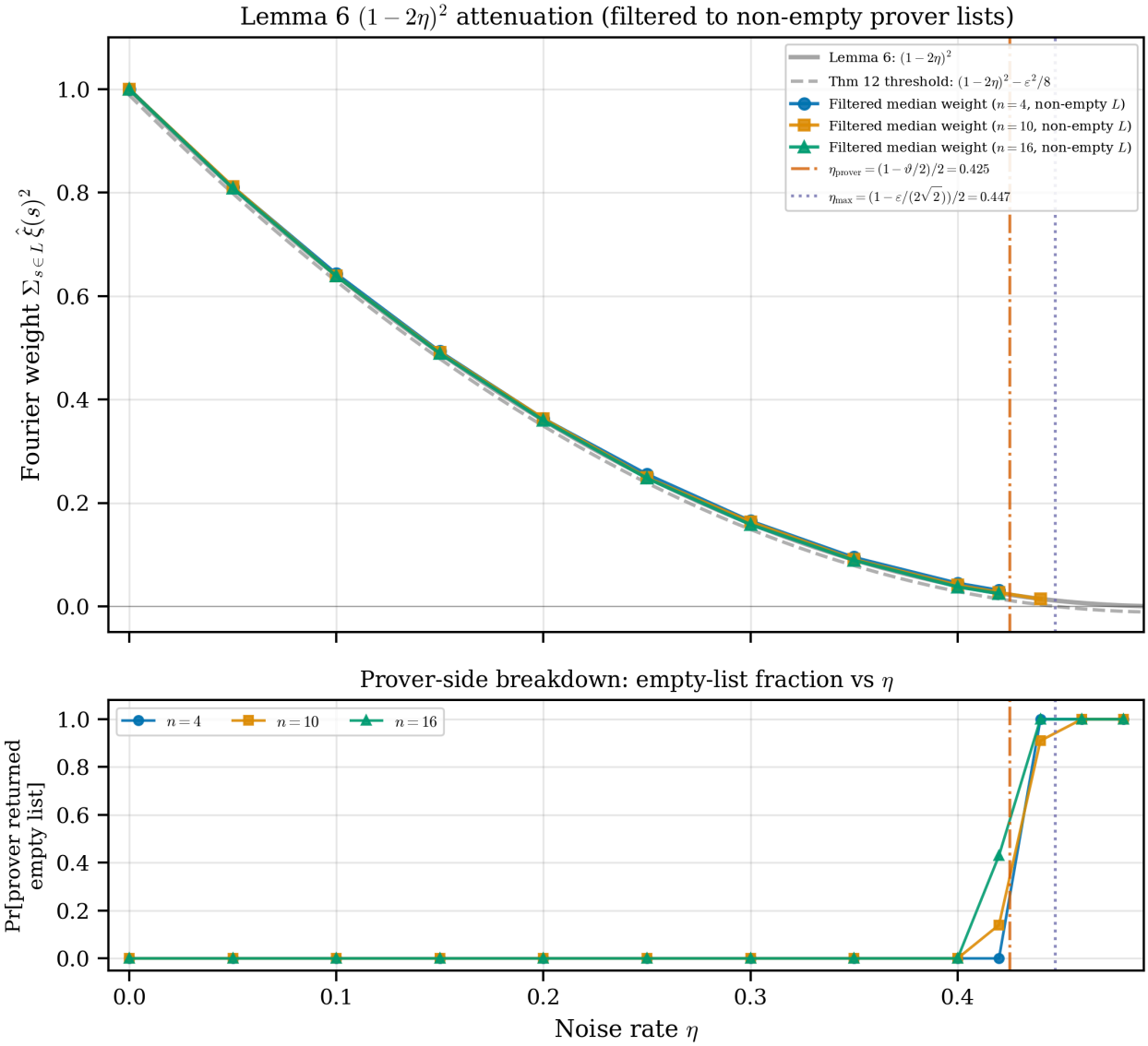

Figure 3: Measured Fourier mass at s^* against the prediction $(1-2\eta)^2$ on single-parity targets.

η	$(1-2\eta)^2$	mean over n	rel. err
0.00	1.0000	1.0000	0.0%
0.05	0.8100	0.8103	0.0%
0.10	0.6400	0.6404	0.1%
0.15	0.4900	0.4894	0.1%
0.20	0.3600	0.3611	0.3%
0.25	0.2500	0.2509	0.4%
0.30	0.1600	0.1615	0.9%
0.35	0.0900	0.0913	1.5%
0.40	0.0400	0.0407	1.8%
0.42	0.0256	0.0258	0.9%

Table 5: Re-decoded filtered median of the QFS mass at s^*, averaged over $n \in \{4, ..., 16\}$ averaged against the $(1-2\eta)^2$ prediction. Relative error stays below 1.8% everywhere on the feasible side of $\eta_{\max}$.

Figure 3 overlays median QFS mass at s^* on the theoretical curve $(1-2\eta)^2$. Table 5 shows the mean over n matches theory to within 1.8% relative error at every $\eta \leq 0.42$.

5.3.2 Gate-level depolarising noise

This experiment probes the robustness of the protocol to per-gate depolarising noise (whilst the verifier samples a noiseless $\rho_{\mathcal{D}}$); unlike every other experiment, Caro et al. [1] provide no theoretical reference point here.

The produced data set `results/gate_noise_4_8_50.pb` contain 50 trials for $n \in \{4, ..., 8\}$, and 12 per-gate error rates with label noise $\eta = 0$:

$$p \in \{0, 0.0001, 0.0005, 0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5\}$$

In this simulation, we force execution to circuit mode, so the oracle is compiled as a gate circuit. This is necessary for per-gate noise simulation, and is why it is infeasible to run this for $n > 8$.

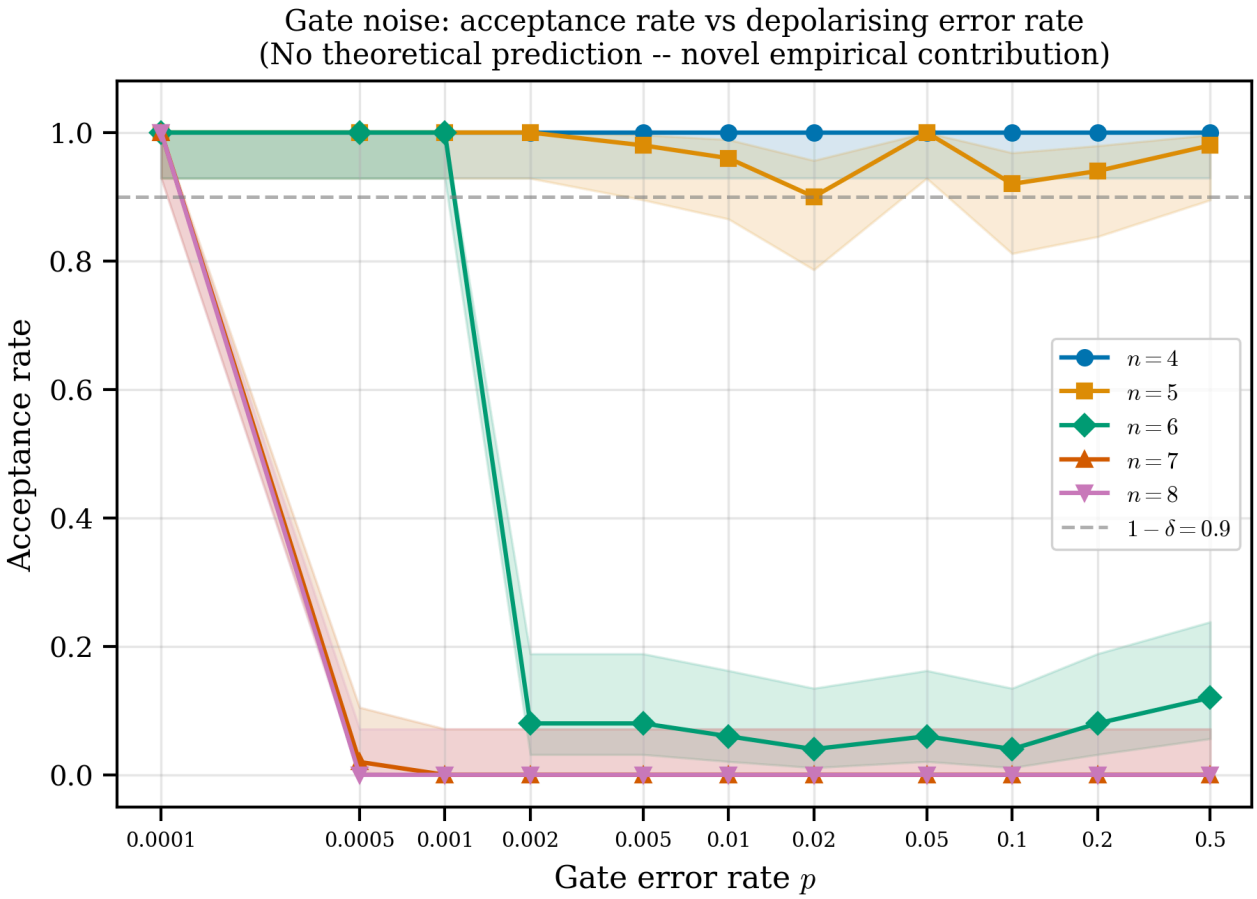

Figure 4: Acceptance rate against prover-side per-gate depolarising noise rate p, broken out by n.

Notice Figure 4 shows sharp degradation to noise p with n, however the protocol appears unexpectedly robust for $n < 6$ and large p. Note the uniform noise floor $\frac{1}{2^n}$ exceeds the extraction threshold $\frac{\vartheta^2}{4} = 0.0225$ precisely when $n \leq 5$, so even a maximally depolarised QFS circuit emits a list that trivially contains the target string. In this case, the list-size bound $\frac{64b^2}{\vartheta^2}$ will not flag for small n.

5.4 Sensitivity and Practical Limitations

The sensitivity experiments characterise how the protocol depends on its tunable parameters, and on the Fourier structure of the target function.

5.4.1 Bent functions

Definition 10 (Bent function). A bent function is a Boolean function $f : \mathbb{F}_2^n \to F_2$ where n is even and achieves maximal nonlinearity. Equivalently, f is bent if and only if its Walsh-Hadamard transform

$$\hat{f}(w) = \sum_{x \in \mathbb{F}_n^2} (-1)^{f(x) \oplus \langle w, x \rangle}$$

satisfies $|\hat{f}(w)| = 2^{\frac{n}{2}}, \forall w \in \mathbb{F}_2^n$. In other words, the function's Fourier spectrum is perfectly flat in magnitude.

The target function in this case is the canonical Maiorana-McFarland bent function $f(x, y) = \langle x, y, \rangle \bmod 2$ over $(\mathbb{F}_2)^{\frac{n}{2}}$, for which $|\hat{g}(s)| = 2^{-(\frac{n}{2})}$ uniformly on all 2^n input strings.

From the conditional probability $\mathbb{P}[s|b = 1]$ in Section 5.3.1, we cam set $\mathbb{E}_{x[\varphi(x)^2]} = 1$ and $\eta = 0$ to collapse the distribution to $\mathbb{P}[s|b = 1] = 2^{-n}$, i.e. exactly uniform and information-theoretically indistinguishable from sampling noise.

From Caro et al. [1, Corollary 5, p.28] we guarantee inclusion of $s \in L$ when $|\hat{g}(s)| \geq \varepsilon$ and excludes s whem $|\hat{g}(s)| < \frac{\varepsilon}{2}$. For bent coefficients $2^{-\frac{n}{2}}$ at $\varepsilon = 0.3$, this locates a crossover at $n = 2 \log_2(\frac{2}{\varepsilon}) \approx 5.47$.

At $n = 4$ all 16 flat-spectrum coefficients clear the conditional QFS extraction threshold mentioned in Section 5.3.2 of 0.0225, so the returned L will contain s^* and V will accept. At $n = 6$, the prover recovers at most one or two spurious coefficients per trial and the verifier rejects.

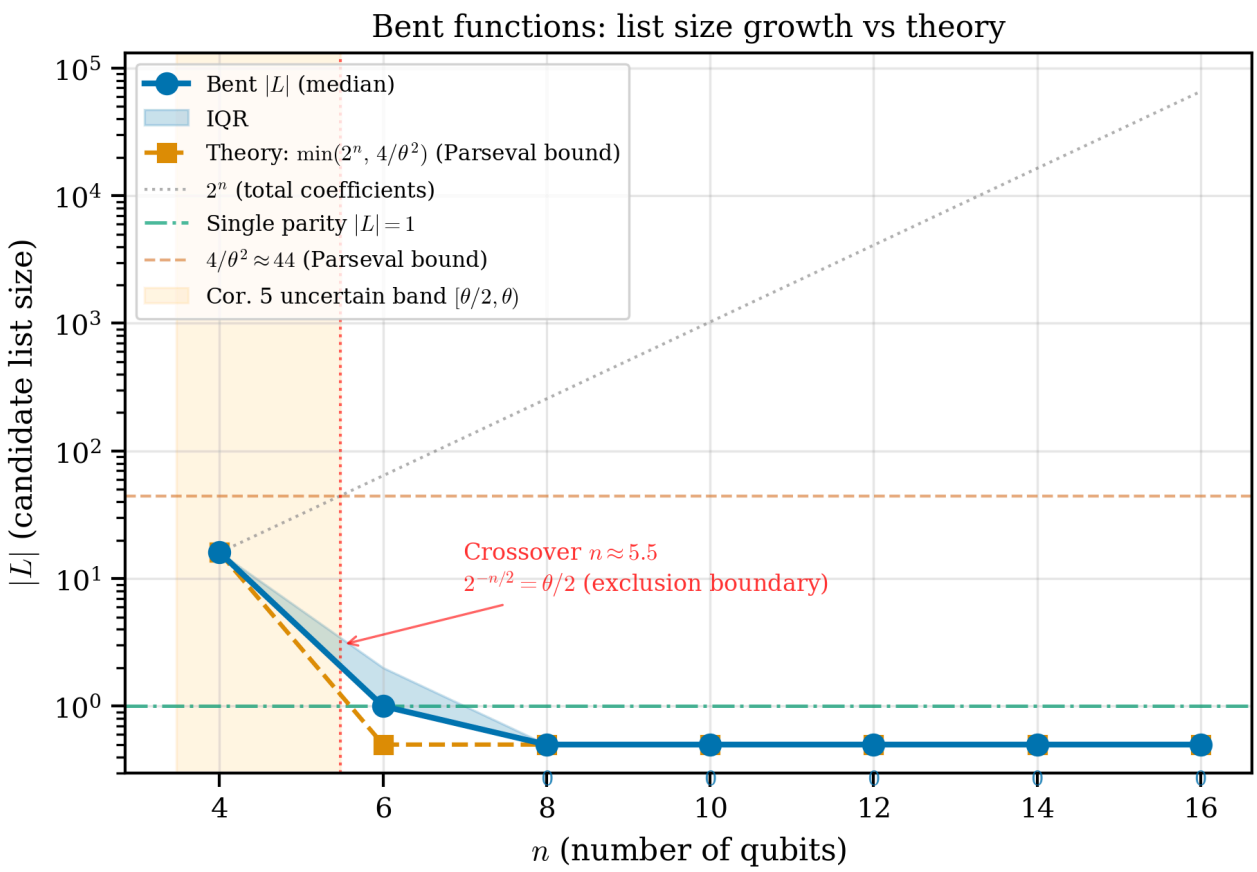

Figure 5: Recovered list size versus n

The sharp breakdown at $n = 6$ in Figure 5 demonstrates this behaviour exactly.

5.4.2 θ-sensitivity

This experiment sweeps the Fourier resolution parameter ϑ against a fixed *sparse plus noise*: a dominant coefficient at a random parity $s^*, c_{\text{dom}} = 0.7$ with three secondary coefficients at $c_{\text{sec}} = 0.1$. To reiterate, the ϑ parameter is precisely responsible for lower-bounding the magnitude of all non-zero Fourier coefficients of the conditional label expectation φ. Every fourier coefficient is either zero or at least ϑ in absolute value.

With this, we can observe the interesting boundary at $\vartheta = 0.2 = 2c_{\text{sec}}$ - the extraction boundary in Figure 6 with the frontier visible as a vertical yellow band.

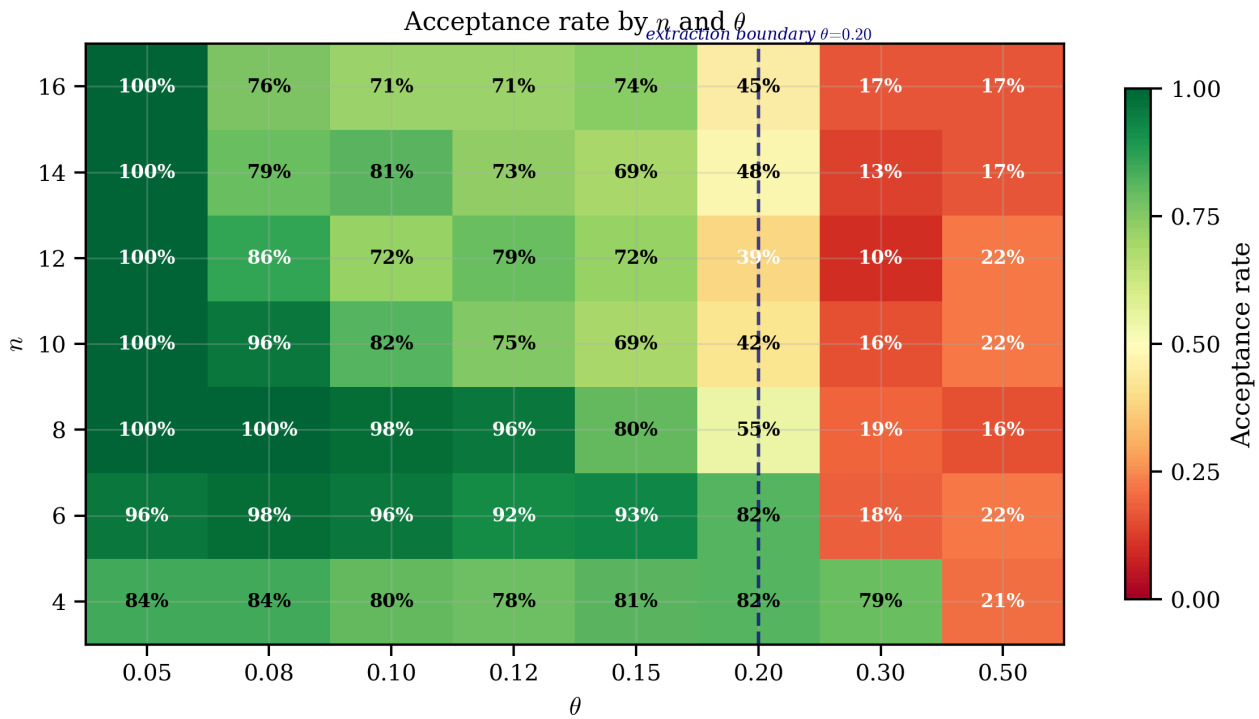

Figure 6: A heatmap of acceptance rate parameterised by n, θ.

Notice that we accept with low probability past the frontier $\vartheta = 0.2$, where φ no longer satisfies the "no small non-zero Fourier coefficients" promise required by the paper, even when the dominant parity is still in hand.

6 Discussion of Experiments

6.1 Alignment with known theory and bounds

6.2 Protocol failure modes

6.3 Gate noise as an open frontier

6.4 Limitations

7 Conclusions and Future Work

7.1 Summary of Contributions

7.2 Future Work

A MoSState Class - An implementation of the MoS concept from Caro et al. [1]

For the sake of conciseness, all comments have been removed from all appendices, but are available in source code.

A.1 State constructor

```
class MoSState:
    def __init__(
        self,
        n: int,
        phi: Union[Callable[[int], float], np.ndarray],
        noise_rate: float = 0.0,
        seed: Optional[int] = None,
    ):
        if n < 1:
            raise ValueError(f"n must be >= 1, got {n}")
        if not 0.0 <= noise_rate <= 0.5:
            raise ValueError(f"noise_rate must be in [0, 0.5], got {noise_rate}")

        self.n: int = n
        self.dim_x: int = 2**n
        self.dim_total: int = 2 ** (n + 1)
        self.noise_rate: float = noise_rate
        self._rng: Generator = default_rng(seed)

        if callable(phi):
            self._phi = np.array(
              [phi(x) for x in range(self.dim_x)], dtype=np.float64
            )
        else:
            self._phi = np.asarray(phi, dtype=np.float64).copy()
            if len(self._phi) != self.dim_x:
                raise ValueError(...)

        if not np.all((self._phi >= 0.0) & (self._phi <= 1.0)):
            raise ValueError("All phi values must be in [0, 1]")

        eta = self.noise_rate
        self._phi_effective: np.ndarray = (1 - 2 * eta) * self._phi + eta

        self._noise_damping: float = 1.0 - 2.0 * eta
```

Listing 15: Constructor for the `MoSState` class.

A.2 Property accessors

```
@property
def phi(self) -> np.ndarray:
    return self._phi

@property
def tilde_phi(self) -> np.ndarray:
    return 1.0 - 2.0 * self._phi

@property
    return self._phi_effective

@property
def tilde_phi_effective(self) -> np.ndarray:
    return 1.0 - 2.0 * self._phi_effective
```

Listing 16: Property accessors for phi in both $\{0, 1\}$ and $\{-1, 1\}$ conventions.

A.3 $f \sim F_{\mathcal{D}}$ sampler

```
def sample_f(self, rng: Optional[Generator] = None) -> np.ndarray:
    if rng is None:
        rng = self._rng

    return (rng.random(self.dim_x) < self._phi_effective).astype(np.uint8)
```

Listing 17: Sampling $f \sim F_{\mathcal{D}}$.

A.4 State preparation

```python
def statevector_f(self, f: np.ndarray) -> Statevector:
    sv_data = np.zeros(self.dim_total, dtype=np.complex128)
    amp = 1.0 / np.sqrt(self.dim_x)

    for x in range(self.dim_x):
        idx = x + int(f[x]) * self.dim_x
        sv_data[idx] = amp

    return Statevector(sv_data)

def _circuit_oracle_f(self, f: np.ndarray) -> QuantumCircuit:
    qr = QuantumRegister(self.n + 1, "q")
    qc = QuantumCircuit(qr, name="oracle_f")

    for x in range(self.dim_x):
        if f[x] == 1:
            ctrl_state = format(x, f"0{self.n}b")
            if self.n == 1:
                qc.cx(0, 1, ctrl_state=ctrl_state)
            else:
                qc.mcx(
                    control_qubits=list(range(self.n)),
                    target_qubit=self.n,
                    ctrl_state=ctrl_state,
                )

    return qc

def circuit_prepare_f(self, f: np.ndarray) -> QuantumCircuit:
    qr = QuantumRegister(self.n + 1, "q")
    qc = QuantumCircuit(qr, name="prepare_psi_f")

    for i in range(self.n):
        qc.h(qr[i])

    oracle = self._circuit_oracle_f(f)
    qc.compose(oracle, inplace=True)

    return qc

def circuit_prepare_f_initialize(self, f: np.ndarray) -> QuantumCircuit:
    sv = self.statevector_f(f)
    qr = QuantumRegister(self.n + 1, "q")
    qc = QuantumCircuit(qr, name="prepare_psi_f_init")
    qc.initialize(sv, qr)
    return qc
```

Listing 18: Both pure state preparation $|\psi_{(\mathcal{U}_n,f)}\rangle$ and circuit preparation for producing $|\psi_{(\mathcal{U}_n,f)}\rangle$.

A.5 Density matrix of $\rho_{\mathcal{D}}$

This is used explicitly for testing and is not seen in experimental code.

```
def density_matrix(
    self,
    num_samples: int = 1000,
    rng: Optional[Generator] = None,
) -> DensityMatrix:
    if rng is None:
        rng = self._rng

    rho_data = np.zeros((self.dim_total, self.dim_total), dtype=np.complex128)

    for _ in range(num_samples):
        f = self.sample_f(rng)
        sv = self.statevector_f(f)
        rho_data += np.outer(sv.data, sv.data.conj())

    rho_data /= num_samples
    return DensityMatrix(rho_data)
```

Listing 19: Approximate density matrix $\rho_{\mathcal{D}}$ by Monte-Carlo.

A.6 Classical sampling mechanisms

```
def sample_classical(
    self,
    rng: Optional[Generator] = None,
) -> Tuple[int, int]:
    if rng is None:
        rng = self._rng

    x = rng.integers(0, self.dim_x)
    y = int(rng.random() < self._phi_effective[x])
    return x, y

def sample_classical_batch(
    self,
    num_samples: int,
    rng: Optional[Generator] = None,
) -> Tuple[np.ndarray, np.ndarray]:
    if rng is None:
        rng = self._rng

    xs = rng.integers(0, self.dim_x, size=num_samples)
    ys = (rng.random(num_samples) < self._phi_effective[xs]).astype(np.uint8)
    return xs, ys
```

Listing 20: Classical sampling tools.

A.7 Fourier analysis tools

```
def fourier_coefficient(self, s: int, *, effective: bool = True) -> float:
    tphi = self.tilde_phi
    parities = np.array([bin(s & x).count("1") % 2 for x in range(self.dim_x)])
    chi_s = 1.0 - 2.0 * parities  # (-1)^{s·x}
    coeff = float(np.mean(tphi * chi_s))
    if effective:
        coeff *= self._noise_damping
    return coeff

def fourier_spectrum(self, *, effective: bool = True) -> np.ndarray:
    tphi = self.tilde_phi
    spectrum = np.empty(self.dim_x, dtype=np.float64)
    for s in range(self.dim_x):
        parities = np.array([bin(s & x).count("1") % 2 for x in range(self.dim_x)])
        chi_s = 1.0 - 2.0 * parities
        spectrum[s] = float(np.mean(tphi * chi_s))
    if effective:
        spectrum *= self._noise_damping
    return spectrum

def parseval_check(self, *, effective: bool = True) -> Tuple[float, float]:
    spectrum = self.fourier_spectrum(effective=effective)
    fourier_sum = float(np.sum(spectrum**2))
    if effective:
        expected_sq = float(np.mean(self.tilde_phi_effective**2))
    else:
        expected_sq = float(np.mean(self.tilde_phi**2))
    return fourier_sum, expected_sq

def qfs_probability(self, s: int) -> float:
    tphi_eff = self.tilde_phi_effective
    E_sq = float(np.mean(tphi_eff**2))
    coeff = self.fourier_coefficient(s, effective=True)
    return (1.0 - E_sq) / self.dim_x + coeff**2

def qfs_distribution(self) -> np.ndarray:
    spectrum_eff = self.fourier_spectrum(effective=True)
    tphi_eff = self.tilde_phi_effective
    E_sq = float(np.mean(tphi_eff**2))
    return (1.0 - E_sq) / self.dim_x + spectrum_eff**2
```

Listing 21: A set of tools for fourier analysis of `MoSState`'s properties.

B The `QuantumFourierSampler` for sampling from MoS

For the sake of conciseness, all comments have been removed from all appendices, but are available in source code.

B.1 Constructor

```python
class QuantumFourierSampler:
  _MODES = {"statevector", "circuit"}

  def __init__(
      self,
      mos_state: MoSState,
      seed: Optional[int] = None,
      noise_model: Optional[object] = None,
  ):
      self.state = mos_state
      self.n = mos_state.n
      self._seed = seed
      self._rng: Generator = default_rng(seed)
      self._noise_model = noise_model
```

Listing 22: Constructor for the sampling MoS wrapper

B.2 Sampling mechanism

```python
def sample(
    self,
    shots: int,
    mode: str = "statevector",
) -> QFSResult:
    if shots < 1:
        raise ValueError(f"shots must be >= 1, got {shots}")
    if mode not in self._MODES:
        raise ValueError(
            f"Unknown mode {mode!r}; expected one of {sorted(self._MODES)}"
        )

    dispatch = {
        "statevector": self._sample_statevector,
        "circuit": self._sample_circuit,
    }
    raw_counts = dispatch[mode](shots)
    ps_counts, ps_shots = self._postselect(raw_counts)

    return QFSResult(
        raw_counts=raw_counts,
        postselected_counts=ps_counts,
        total_shots=shots,
        postselected_shots=ps_shots,
        n=self.n,
        mode=mode,
    )

def theoretical_distribution(self) -> np.ndarray:
    return self.state.qfs_distribution()

def fourier_coefficient(
    self,
    s: int,
    effective: bool = True,
) -> float:
    return self.state.fourier_coefficient(s, effective=effective)
```

Listing 23: Exposed sampling and state functions

B.2.1 Inner sampling mechanism

```python
def _sample_statevector(self, shots: int) -> dict[str, int]:
    n = self.n
    dim_total = self.state.dim_total
    counts: dict[str, int] = {}

    h_circuit = QuantumCircuit(n + 1, name="H_all")
    for q in range(n + 1):
        h_circuit.h(q)

    for _ in range(shots):
        f = self.state.sample_f(rng=self._rng)
        psi_f = self.state.statevector_f(f)
        psi_h = psi_f.evolve(h_circuit)

        probs = psi_h.probabilities()
        idx = self._rng.choice(dim_total, p=probs)
        bitstring = format(idx, f"0{n + 1}b")
        counts[bitstring] = counts.get(bitstring, 0) + 1

    return counts

def _postselect(
    self,
    raw_counts: dict[str, int],
) -> tuple[dict[str, int], int]:
    ps_counts: dict[str, int] = {}
    ps_total = 0
    for bitstring, count in raw_counts.items():
        if bitstring[0] == "1":
            s_bits = bitstring[1:]
            ps_counts[s_bits] = ps_counts.get(s_bits, 0) + count
            ps_total += count
    return ps_counts, ps_total
```

Listing 24: Statevector-based sampling and postselection logic

```python
def _sample_circuit(self, shots: int) -> dict[str, int]:
    if self.n > 12:
        warnings.warn(...)

    n = self.n
    counts: dict[str, int] = {}

    circuits = []
    for _ in range(shots):
        f = self.state.sample_f(rng=self._rng)
        qc = self.state.circuit_prepare_f(f)
        for q in range(n + 1):
            qc.h(q)
        qc.measure_all()
        circuits.append(qc)

    if self._noise_model is not None:
        from qiskit_aer import AerSimulator
        from qiskit import transpile

        backend = AerSimulator(noise_model=self._noise_model)
        for qc in circuits:
            child_seed = int(self._rng.integers(0, 2**31))
            qc_t = transpile(qc, backend)
            result = backend.run(
                qc_t, shots=1, seed_simulator=child_seed
            ).result()
            for bitstring, cnt in result.get_counts().items():
                # AerSimulator may include spaces in bitstrings;
                # strip them for consistency.
                bs = bitstring.replace(" ", "")
                counts[bs] = counts.get(bs, 0) + cnt
    else:
        child_rng = default_rng(int(self._rng.integers(0, 2**31)))
        sampler = StatevectorSampler(seed=child_rng)
        job = sampler.run(circuits, shots=1)
        for pub_result in job.result():
            for bitstring, cnt in pub_result.data.meas.get_counts().items():
                counts[bitstring] = counts.get(bitstring, 0) + cnt

    return counts
```

Listing 25: Circuit-based (slower) sampling mechanism

B.3 QFSResult class

```
@dataclass(frozen=True)
class QFSResult:
    raw_counts: dict[str, int]
    postselected_counts: dict[str, int]
    total_shots: int
    postselected_shots: int
    n: int
    mode: str

    @property
    def postselection_rate(self) -> float:
        if self.total_shots == 0:
            return 0.0
        return self.postselected_shots / self.total_shots

    def empirical_distribution(self) -> np.ndarray:
        dim = 2**self.n
        dist = np.zeros(dim, dtype=np.float64)
        if self.postselected_shots == 0:
            return dist
        for bitstring, count in self.postselected_counts.items():
            s = int(bitstring, 2)
            dist[s] += count
        dist /= self.postselected_shots
        return dist
```

Listing 26: Compacted results from the Fourier sampling subroutine.

C Implementation of the Prover from Caro et al. [1]

For the sake of conciseness, all comments have been removed from all appendices, but are available in source code.

C.1 Approximation result class

```
@dataclass(frozen=True)
class SpectrumApproximation:
    entries: dict[int, float]
    threshold: float
    n: int
    num_qfs_samples: int
    total_qfs_shots: int
```

Listing 27: Succinct approximation of a Fourier spectrum.

C.2 Prover's message to the verifier

```
@dataclass(frozen=True)
class ProverMessage:
    L: list[int]
    estimates: dict[int, float]
    n: int
    epsilon: float
    theta: float
    spectrum_approx: SpectrumApproximation
    qfs_result: QFSResult
    num_classical_samples: int

    @property
    def list_size(self) -> int:
        """Number of candidate heavy coefficients."""
        return len(self.L)

    @property
    def total_copies_used(self) -> int:
        """Total MoS copies consumed (QFS + classical estimation)."""
        return self.spectrum_approx.total_qfs_shots + self.num_classical_samples
```

Listing 28: The message the prover sends to the verifier in Caro et al. [1], with some extra information for debugging, testing and analysis.

C.3 Constructor of `MoSProver`

```
class MoSProver:
    def __init__(
        self,
        mos_state: MoSState,
        seed: Optional[int] = None,
        noise_model: Optional[object] = None,
    ):
        self.state = mos_state
        self.n = mos_state.n
        self._seed = seed
        self._rng: Generator = default_rng(seed)
        self._noise_model = noise_model
```

Listing 29: Constructor for the MoSProver

C.4 The central running protocol for sampling and aggregation of $\rho_{\mathcal{D}}$

```
def run_protocol(
    self,
    epsilon: float,
    delta: float = 0.1,
    theta: Optional[float] = None,
    estimate_coefficients: bool = True,
    qfs_mode: str = "statevector",
    qfs_shots: Optional[int] = None,
    classical_samples: Optional[int] = None,
) -> ProverMessage:
    # Source code contains parameter validation here, we omit for conciseness.

    tau = theta**2 / 8.0
    if qfs_shots is None:
        m_postselected = int(np.ceil(2.0 * np.log(4.0 / delta) / tau**2))
        qfs_shots = int(np.ceil(2.5 * m_postselected))
    sampler = QuantumFourierSampler(
        self.state,
        seed=int(self._rng.integers(0, 2**31)),
        noise_model=self._noise_model,
    )
    qfs_result = sampler.sample(shots=qfs_shots, mode=qfs_mode)
    spectrum_approx = self._build_spectrum_approximation(
        qfs_result=qfs_result,
        theta=theta,
    )
    L = self._extract_heavy_list(
        spectrum_approx=spectrum_approx,
        theta=theta,
    )
    estimates: dict[int, float] = {}
    num_classical = 0
    if estimate_coefficients and len(L) > 0:
        estimates, num_classical = self._estimate_coefficients(
            L=L,
            epsilon=epsilon,
            delta=delta,
            num_samples_override=classical_samples,
        )
    return ProverMessage(
        ...
    )
```

Listing 30: The central running protocol from Corollary 1.

C.5 Building a Fourier spectrum approximation

```
def _build_spectrum_approximation(
    self,
    qfs_result: QFSResult,
    theta: float,
) -> SpectrumApproximation:
    n = self.n
    ps_total = qfs_result.postselected_shots

    extraction_threshold = theta**2 / 4.0

    entries: dict[int, float] = {}
    if ps_total > 0:
        for bitstring, count in qfs_result.postselected_counts.items():
            s = int(bitstring, 2)
            empirical_prob = count / ps_total
            if empirical_prob >= extraction_threshold:
                entries[s] = empirical_prob

    return SpectrumApproximation(
        entries=entries,
        threshold=extraction_threshold,
        n=n,
        num_qfs_samples=ps_total,
        total_qfs_shots=qfs_result.total_shots,
    )
```

Listing 31: Build the spectrum approximation from sample results.

C.6 Estimating and extracting heavy coefficients

```
def _extract_heavy_list(
    self,
    spectrum_approx: SpectrumApproximation,
    theta: float,
) -> list[int]:
    L = sorted(
        spectrum_approx.entries.keys(),
        key=lambda s: spectrum_approx.entries[s],
        reverse=True,
    )

    parseval_bound = int(np.ceil(16.0 / theta**2))
    if len(L) > parseval_bound:
        L = L[:parseval_bound]

    return L
```

Listing 32: Extracting heavy coefficients and list truncation from the `SpectrumApproximation`.

```python
def _estimate_coefficients(
    self,
    L: list[int],
    epsilon: float,
    delta: float,
    num_samples_override: Optional[int] = None,
) -> tuple[dict[int, float], int]:
    L_size = len(L)
    if L_size == 0:
        return {}, 0
    if num_samples_override is not None:
        num_samples = num_samples_override
    else:
        num_samples = int(np.ceil(2.0 / epsilon**2 * np.log(4.0 * L_size / delta)))
        num_samples = max(num_samples, 100)

    xs, ys = self.state.sample_classical_batch(
        num_samples=num_samples,
        rng=self._rng,
    )
    signed_labels = 1.0 - 2.0 * ys.astype(np.float64)
    estimates: dict[int, float] = {}
    for s in L:
        parities = np.array(
            [bin(s & int(x)).count("1") % 2 for x in xs],
            dtype=np.float64,
        )
        chi_s = 1.0 - 2.0 * parities
        est = float(np.mean(signed_labels * chi_s))
        est = np.clip(est, -1.0, 1.0)
        estimates[s] = est

    return estimates, num_samples

def exact_heavy_coefficients(
    self,
    theta: float,
    *,
    effective: bool = True,
) -> list[tuple[int, float]]:
    spectrum = self.state.fourier_spectrum(effective=effective)
    heavy = [
        (s, float(spectrum[s]))
        for s in range(self.state.dim_x)
        if abs(spectrum[s]) >= theta
    ]
    heavy.sort(key=lambda t: abs(t[1]), reverse=True)
    return heavy
```

Listing 33: Fourier coefficient estimation.

D Implementation of the Verifier from Caro et al. [1]

For the sake of conciseness, all comments have been removed from all appendices, but are available in source code. The file we describe in this appendix `ql/verifier.py` is especially large, so we omit less important code.

D.1 Verification outcomes and hypothesis types

```python
class VerificationOutcome(Enum):
    ACCEPT = "accept"
    REJECT_LIST_TOO_LARGE = "reject_list_too_large"
    REJECT_INSUFFICIENT_WEIGHT = "reject_insufficient_weight"

class HypothesisType(Enum):
    PARITY = "parity"
    FOURIER_SPARSE = "fourier_sparse"
```

Listing 34: Classes representing outcomes of the protocol and hypotheses over Caro et al. [1, Theorem 12, 15].

D.2 Parity hypothesis for 1-agnostic learning

```python
@dataclass(frozen=True)
class ParityHypothesis:
    s: int
    n: int
    estimated_coefficient: float

    def evaluate(self, x: int) -> int:
        return bin(self.s & x).count("1") % 2

    def evaluate_batch(self, xs: np.ndarray) -> np.ndarray:
        return np.array(
            [bin(self.s & int(x)).count("1") % 2 for x in xs],
            dtype=np.uint8,
      )
```

Listing 35: Output hypothesis of the protocol from [1, Theorem 12].

D.3 Fourier-k-sparse hypothesis for 2-agnostic learning

```
@dataclass(frozen=True)
class FourierSparseHypothesis:
    coefficients: dict[int, float]
    n: int

    def g(self, x: int) -> float:
        val = 0.0
        for s, coeff in self.coefficients.items():
            parity = bin(s & x).count("1") % 2
            chi_s = 1.0 - 2.0 * parity
            val += coeff * chi_s
        return val

    def evaluate(self, x: int, rng: Optional[Generator] = None) -> int:
        if rng is None:
            rng = default_rng()
        gx = self.g(x)
        p = (1.0 - gx) ** 2 / (2.0 * (1.0 + gx**2))
        p = np.clip(p, 0.0, 1.0)
        return int(rng.random() < p)

    def evaluate_batch(
        self, xs: np.ndarray, rng: Optional[Generator] = None
    ) -> np.ndarray:
        if rng is None:
            rng = default_rng()
        return np.array([self.evaluate(int(x), rng=rng) for x in xs],
dtype=np.uint8)
```

Listing 36: Output hypothesis of the protocol from [1, Theorem 15].

D.4 Verification result

```
@dataclass(frozen=True)
class VerificationResult:
    outcome: VerificationOutcome
    hypothesis: Optional[Union[ParityHypothesis, FourierSparseHypothesis]]
    hypothesis_type: Optional[HypothesisType]
    verifier_estimates: dict[int, float]
    accumulated_weight: float
    acceptance_threshold: float
    list_received: list[int]
    list_size_bound: int
    n: int
    epsilon: float
    num_classical_samples: int

    @property
    def accepted(self) -> bool:
        return self.outcome == VerificationOutcome.ACCEPT
```

Listing 37: Complete protocol result summary.

D.5 MoSVerifier Constructor

```
class MoSVerifier:
    def __init__(
        self,
        mos_state: MoSState,
        seed: Optional[int] = None,
    ):
        self.state = mos_state
        self.n = mos_state.n
        self._seed = seed
        self._rng: Generator = default_rng(seed)
```

Listing 38: Constructor for the `MoSVerifier` class.

D.6 General verifier logic

```
def _verify_core(
    self,
    prover_message: ProverMessage,
    epsilon, delta, theta, a_sq, b_sq,
    hypothesis_type: HypothesisType,
    k: Optional[int],
    num_samples: int = 0,
) -> VerificationResult:
    n = self.n
    L = prover_message.L
    list_size_bound = int(np.ceil(64.0 * b_sq / theta**2))
    if len(L) > list_size_bound:
        return VerificationResult(
            outcome=VerificationOutcome.REJECT_LIST_TOO_LARGE, ...
        )
    L_size = len(L)
    if hypothesis_type == HypothesisType.PARITY:
        per_coeff_tolerance = epsilon**2 / (16.0 * max(L_size, 1))
    else:
        per_coeff_tolerance = epsilon**2 / (256.0 * k**2 * max(L_size, 1))
    if L_size > 0:
        num_samples = int(
            np.ceil(2.0 / per_coeff_tolerance**2 * np.log(4.0 * L_size / delta))
        )
        num_samples = max(num_samples, 100)
    verifier_estimates = self._estimate_coefficients_independently(
        L=L,
        num_samples=num_samples,
    )
    accumulated_weight = sum(verifier_estimates.get(s, 0.0) ** 2 for s in L)
    if hypothesis_type == HypothesisType.PARITY:
        acceptance_threshold = a_sq - epsilon**2 / 8.0
    else:
        acceptance_threshold = a_sq - epsilon**2 / (128.0 * k**2)
    if accumulated_weight < acceptance_threshold:
        return VerificationResult(
            outcome=VerificationOutcome.REJECT_INSUFFICIENT_WEIGHT, ...
        )
    if hypothesis_type == HypothesisType.PARITY:
        hypothesis = self._build_parity_hypothesis(L, verifier_estimates)
    else:
        hypothesis = self._build_fourier_sparse_hypothesis(L, verifier_estimates,
k)
    return VerificationResult(
        outcome=VerificationOutcome.ACCEPT, ...
    )
```

Listing 39: Verification logic from Theorem 3.

D.7 Verifier coefficient estimation

```python
def _estimate_coefficients_independently(
    self,
    L: list[int],
    num_samples: int,
) -> dict[int, float]:
    if len(L) == 0 or num_samples == 0:
        return {s: 0.0 for s in L}

    xs, ys = self.state.sample_classical_batch(
        num_samples=num_samples,
        rng=self._rng,
    )

    signed_labels = 1.0 - 2.0 * ys.astype(np.float64)

    estimates: dict[int, float] = {}
    for s in L:
        parities = np.array(
            [bin(s & int(x)).count("1") % 2 for x in xs],
            dtype=np.float64,
        )
        chi_s = 1.0 - 2.0 * parities
        est = float(np.mean(signed_labels * chi_s))
        est = np.clip(est, -1.0, 1.0)
        estimates[s] = est

    return estimates
```

Listing 40: Classical Fourier coefficient estimator for the verifier.

D.8 Building hypotheses

```python
def _build_parity_hypothesis(
    self,
    L: list[int],
    verifier_estimates: dict[int, float],
) -> ParityHypothesis:
    if not L:
        return ParityHypothesis(s=0, n=self.n, estimated_coefficient=0.0)

    s_out = max(L, key=lambda s: abs(verifier_estimates.get(s, 0.0)))
    return ParityHypothesis(
        s=s_out,
        n=self.n,
        estimated_coefficient=verifier_estimates.get(s_out, 0.0),
    )

def _build_fourier_sparse_hypothesis(
    self,
    L: list[int],
    verifier_estimates: dict[int, float],
    k: int,
) -> FourierSparseHypothesis:
    sorted_L = sorted(
        L, key=lambda s: abs(verifier_estimates.get(s, 0.0)), reverse=True
    )

    top_k = sorted_L[:k]
    coefficients = {s: verifier_estimates.get(s, 0.0) for s in top_k}
    return FourierSparseHypothesis(coefficients=coefficients, n=self.n)
```

Listing 41: Functions to build hypotheses for [1, Thm. 12, 15].

E Circuit diagram of sampling MoS with $n = 3$

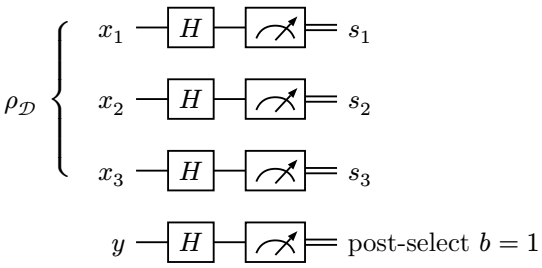

Figure 7: **Mixture-of-superpositions quantum Fourier sampling.** A single copy of the $(n+1)$ -qubit mixed state ρ_D is prepared (left block), Hadamard gates H are applied to every qubit, and all qubits are measured in the computational basis. The label qubit (y) is post-selected on outcome $b = 1$, which succeeds with probability $1/2$. The remaining n-bit string $s = (s_1, ..., s_n)$ yields an approximate Fourier sample of $\varphi = 1 - 2\phi$.

Bibliography

[1] M. C. Caro, M. Hinsche, M. Ioannou, A. Nietner, and R. Sweke, "Classical Verification of Quantum Learning," Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 24:1–24:23. doi: 10.4230/LIPICS.ITCS.2024.24.

[2] P. A. M. Dirac, "A new notation for quantum mechanics," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 35, no. 3, pp. 416–418, 1939, doi: 10.1017/S0305004100021162.

[3] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," in *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, in STOC '85. Providence, Rhode Island, USA: Association for Computing Machinery, 1985, pp. 291–304. doi: 10.1145/22145.22178.

[4] A. Shamir, "IP = PSPACE," *J. ACM*, vol. 39, no. 4, pp. 869–877, Oct. 1992, doi: 10.1145/146585.146609.

[5] D. Haussler, "Decision theoretic generalizations of the PAC model for neural net and other learning applications," *Information and Computation*, vol. 100, no. 1, pp. 78–150, 1992, doi: https://doi.org/10.1016/0890-5401(92)90010-D.

[6] N. H. Bshouty and J. C. Jackson, "Learning DNF over the uniform distribution using a quantum example oracle," in *Proceedings of the Eighth Annual Conference on Computational Learning Theory*, in COLT '95. Santa Cruz, California, USA: Association for Computing Machinery, 1995, pp. 118–127. doi: 10.1145/225298.225312.

[7] R. O'Donnell, "Analysis of Boolean Functions." [Online]. Available: https://arxiv.org/abs/2105.10386

[8] O. Goldreich and L. A. Levin, "A hard-core predicate for all one-way functions," in *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, in STOC '89. Seattle, Washington, USA: Association for Computing Machinery, 1989, pp. 25–32. doi: 10.1145/73007.73010.

[9] E. Bernstein and U. Vazirani, "Quantum complexity theory," in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*, in STOC '93. San Diego, California, USA: Association for Computing Machinery, 1993, pp. 11–20. doi: 10.1145/167088.167097.

[10] A. Dvoretzky, J. Kiefer, and J. Wolfowitz, "Asymptotic Minimax Character of the Sample Distribution Function and of the Classical Multinomial Estimator," *The Annals of Mathematical Statistics*, vol. 27, no. 3, pp. 642–669, 1956, Accessed: Apr. 07, 2026. [Online]. Available: http://www.jstor.org/stable/2237374

[11] S. Goldwasser, G. N. Rothblum, J. Shafer, and A. Yehudayoff, "Interactive Proofs for Verifying Machine Learning," in *12th Innovations in Theoretical Computer Science Conference (ITCS 2021)*, J. R. Lee, Ed., in Leibniz International Proceedings in Infor-

matics (LIPIcs), vol. 185. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 41:1–41:19. doi: 10.4230/LIPIcs.ITCS.2021.41.

[12] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit." 2024. doi: 10.48550/arXiv.2405.08810.